摘要

本报告围绕虚拟现实课程的作业展开。首先，针对我小组课堂汇报中的人脸分析与识别主题，完成了浏览器端人脸分析项目的详细讲解和 dlib 人脸识别示例的解析。基于 face-api.js，分析了静态网页中模型加载、图片输入、摄像头视频流处理与 Canvas 可视化流程，实现了人脸检测、关键点定位、表情识别以及年龄性别估计；结合 dlib 官方度量学习示例，进一步剖析了训练数据组织、mini-batch 采样、ResNet 描述子网络、loss_metric 损失函数与距离阈值验证机制，阐明人脸描述子模型如何通过特征距离完成身份比较。此外，在完成课程要求的基础上，本文还开展了面向三维虚拟场景搭建的附加实践：使用 Unity VR 模板构建森林场景，导入森林、花卉与动物资源，修复旧资源在 Unity 6 + URP 环境下的粉色材质问题，接入 XR Origin 与移动系统，并完成 Windows 平台的打包运行。报告涵盖了模型推理、工程实现、资源兼容与场景发布等课程实践内容。

关键词：人脸分析；人脸识别；face-api.js；dlib；度量学习；Unity；虚拟现实；URP

ABSTRACT

This report summarizes the coursework completed for the Virtual Reality course. First, based on the topic of face analysis and recognition presented by my group in class, it provides a detailed explanation of a browser-based face analysis project and analyzes a dlib face recognition example. Using face-api.js, the report examines the workflow of model loading, image input, camera video stream processing, and Canvas-based visualization in a static web page, implementing face detection, facial landmark localization, expression recognition, and age and gender estimation. It also discusses dlib's official metric learning example, including training data organization, mini-batch sampling, the ResNet descriptor network, the loss_metric loss function, and distance-threshold verification, explaining how a face descriptor model compares identities through feature distances. In addition to the required coursework, this report includes an extended practice project on constructing a three-dimensional virtual scene. A forest scene was built with the Unity VR template, forest, flower, and animal assets were imported, pink material issues caused by legacy assets in Unity 6 with URP were resolved, XR Origin and locomotion systems were integrated, and the project was packaged for Windows. Overall, the report covers practical work in model inference, engineering implementation, asset compatibility, and scene deployment.

Keywords: face analysis; face recognition; face-api.js; dlib; metric learning; Unity; virtual reality; URP

目　　录

1　基于 face-api.js 的浏览器端人脸分析实验　1

1.1　实验定位与内容概述　1

1.2　项目结构与资源组织　2

1.2.1　静态项目结构　2

1.2.2　模型权重与本地加载　2

1.2.3　页面布局与显示容器　4

1.3　脚本加载与初始化流程　4

1.3.1　defer 脚本顺序　4

1.3.2　模型加载的并行等待　5

1.4　图片输入处理流程　5

1.4.1　从本地文件到图像对象　5

1.4.2　检测结果的尺寸映射　6

1.5　表情识别模块分析　7

1.5.1　模型调用链　7

1.5.2　结果排序与概率展示　7

1.5.3　实验现象与局限　8

1.6　年龄与性别估计模块分析　8

1.6.1　模型调用链与输出字段　8

1.6.2　Canvas 标注与结果卡片　8

1.6.3　结果解释与伦理边界　8

1.7　摄像头实时追踪模块分析　10

1.7.1　摄像头授权与视频流绑定　10

1.7.2　周期检测与画布同步　10

1.7.3　结果节流与状态管理　11

1.8　实验复现步骤与观察记录　12

1.8.1　页面访问与资源检查　12

1.8.2　HTML 结构与脚本变量对应　12

1.8.3　建议记录的实验现象　13

1.9　异常场景与边界条件　13

1.9.1　模型文件加载失败　13

1.9.2　输入为空或格式异常　14

1.9.3 摄像头权限与安全上下文 14
1.10 评价指标与实验扩展 14
1.10.1 检测类指标 14
1.10.2 属性估计指标 15
1.10.3 实时性能指标 15
1.11 综合实验结果与分析 15
1.11.1 功能完整性 15
1.11.2 性能影响因素 15
1.11.3 可靠性与数据质量 16
1.12 隐私、安全与改进方向 16
1.13 本章小结 17
参考文献 **19**
附录 实验关键代码 **21**
页面入口与展示容器 21
图片表情识别脚本 23
年龄与性别估计脚本 26
摄像头实时追踪脚本 28
人脸匹配与参考样本加载代码 33
开源人脸识别模型训练代码 34
Unity 森林场景 URP Shader 修复代码 40
Unity VR 移动系统关键代码 44

1 基于 face-api.js 的浏览器端人脸分析实验

1.1 实验定位与内容概述

本章分析的是一个基于 face-api.js 的浏览器端人脸分析开源项目。我们小组的这个项目代码托管在我的 github 仓库 git@github.com:glance02/face-recognition.git 上。这个仓库是在 willtrinh/face-recognition.git 项目基础上 fork 并修改得到的实验版本。项目以 HTML、CSS 和 JavaScript 静态资源为主体，通过 face-api.js[1] 在浏览器中加载预训练模型，并在本地完成图片表情识别、图片年龄与性别估计以及摄像头实时人脸追踪。它的重点不是训练新的神经网络，而是观察一个已经训练好的模型体系如何被前端页面组织、加载、调用和展示。

页面中的 models/ 目录保存了实验所需的预训练权重，public/ 目录提供示例图片、人物参考图片和演示视频等资源。运行时，浏览器脚本先加载模型文件，再把用户上传的图片或摄像头视频帧送入模型，最后将检测框、关键点、表情类别、年龄和性别估计等结果绘制到页面上。整个过程体现了前端页面如何把输入采集、模型推理和结果可视化连接成一个完整的人脸分析流程。

人脸分析是计算机视觉在人机交互、身份认证、内容理解和智能终端中的典型任务。传统系统常把模型部署在服务器端，客户端只负责采集图像、上传数据和显示结果。这种方式便于集中管理模型，但会带来网络依赖、数据传输、服务部署和延迟控制等问题。随着 TensorFlow.js 等前端推理框架的发展，浏览器不仅能显示页面，也能成为模型推理运行环境。这种部署方式具有明显优势：运行入口直观、依赖环境轻、实验过程容易展示，也便于把“页面结构”和“识别流程”对应起来分析。

本实验选择三个页面作为观察对象：face_expression_recognition.html 用于图片表情识别，age_gender_estimation.html 用于年龄与性别估计，webcam_face_tracking.html 用于摄像头实时追踪。三个页面共享同一个模型目录和前端资源目录，但各自加载不同业务脚本。通过对它们进行对比，可以看出浏览器端人脸分析系统的一般流程：先加载模型，再取得输入，再执行模型推理，最后把检测框、关键点、表情或属性结果渲染到界面上。

本章的实验目标包括四个方面。第一，梳理静态网页项目中页面、样式、脚本、模型权重和示例资源之间的组织关系；第二，分析图片输入与摄像头视频流输入在前端实现上的差异；第三，分别说明表情识别、年龄性别估计和实时追踪三个功能模块的实现逻辑；第四，总结浏览器端推理在性能、稳定性、隐私和实验复现方面

的优点与局限。

1.2 项目结构与资源组织

1.2.1 静态项目结构

该项目没有后端服务，也没有复杂的构建入口。页面入口直接位于项目根目录，业务脚本位于 `dist/`，模型文件位于 `models/`，示例图片、人物注册图片和演示视频位于 `public/`。从实验复现角度看，这种结构的好处是入口清晰：只要浏览器能够访问静态文件，就可以加载页面并触发识别逻辑；对于摄像头页面，还需要浏览器允许页面访问本机摄像头。

项目中的核心库文件是 `dist/face-api.min.js`。它是 `face-api.js` 的浏览器端构建文件，向页面暴露全局对象 `faceapi`。三个实验页面并不直接实现底层神经网络，而是调用这个对象提供的高层接口，例如模型加载接口、检测接口、关键点接口、表情接口和绘图接口。业务脚本则负责把这些接口连接成完整实验流程。

页面、脚本和功能之间的对应关系如表 1.1 所示。

表 1.1 浏览器端人脸分析实验页面与脚本对应关系

页面文件	业务脚本	主要功能
face_expression_recognition.html	dist/expression_script.js	图片表情识别
age_gender_estimation.html	dist/estimation_script.js	年龄与性别估计
webcam_face_tracking.html	dist/webcam_script.js	摄像头实时追踪

1.2.2 模型权重与本地加载

`models/` 目录中保存了多个预训练模型权重，包括 Tiny Face Detector、SSD MobileNetV1、68 点人脸关键点模型、表情识别模型、年龄性别估计模型和人脸识别模型等。业务脚本中的 `MODEL_URL` 均设置为 `"./models"`，说明当前实验版本从本地目录加载模型，而不是依赖远程 CDN 或 GitHub Raw URL。

本地模型加载对课程报告很重要。首先，它减少了外部网络波动对实验复现的影响。其次，它使报告中的路径说明与实际项目文件保持一致，读者可以直接在仓库中找到模型、脚本和示例资源。最后，本地加载也更容易观察模型文件数量和页面请求过程，有助于理解前端深度学习应用并不是只有一个 HTML 页面，而是由页面、模型权重、脚本和媒体资源共同组成。

需要注意的是，本地保存权重不等于本地训练模型。浏览器页面运行时只通过 `loadFromUri` 把权重读入内存，然后执行前向推理。无论用户上传多少图片，模型

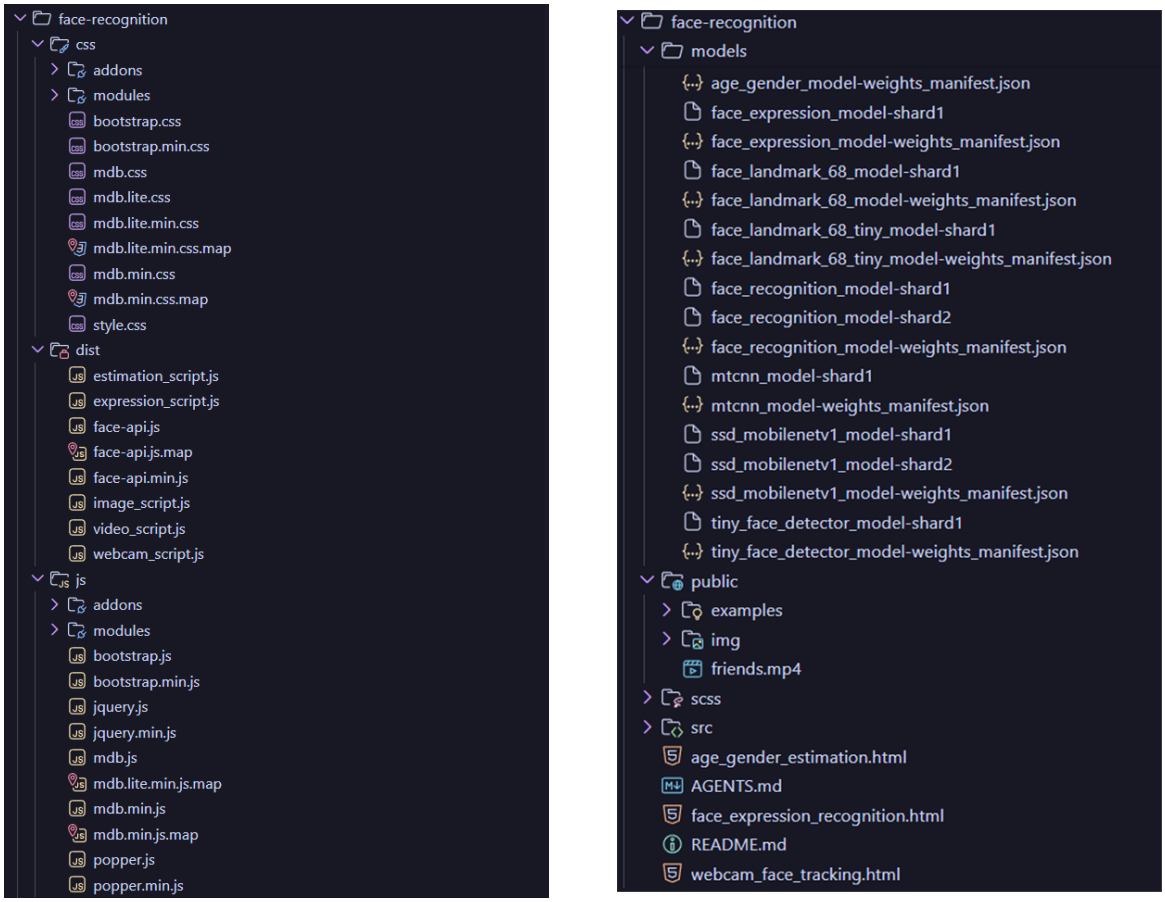


图 1.1　浏览器端人脸分析项目目录结构

参数都不会被更新。这个边界决定了本章的分析口径：页面展示的是“如何使用模型”，不是“如何训练模型”。

1.2.3 页面布局与显示容器

三个页面采用相似的界面组织方式：顶部为导航区域，中间主体为左右分栏。左侧通常放置文件上传控件、摄像头状态、识别结果卡片和空状态提示；右侧放置图片或视频展示区域，并在媒体元素上叠加 Canvas 画布。这样的布局适合实验观察，因为图像标注和结构化结果可以并排出现。用户既能看到检测框是否对准人脸，也能读取模型输出的表情概率、年龄估计、性别置信度等信息。

从职责分工看，HTML 负责定义页面结构，CSS 负责布局和视觉样式，JavaScript 负责模型加载、输入处理、推理调用和结果渲染。页面本身不包含复杂算法，算法能力主要来自 `face-api.js` 和预训练模型。业务脚本则处在连接层：它把 DOM 输入转成模型可处理的媒体对象，把模型输出转成 Canvas 标注和结果卡片。

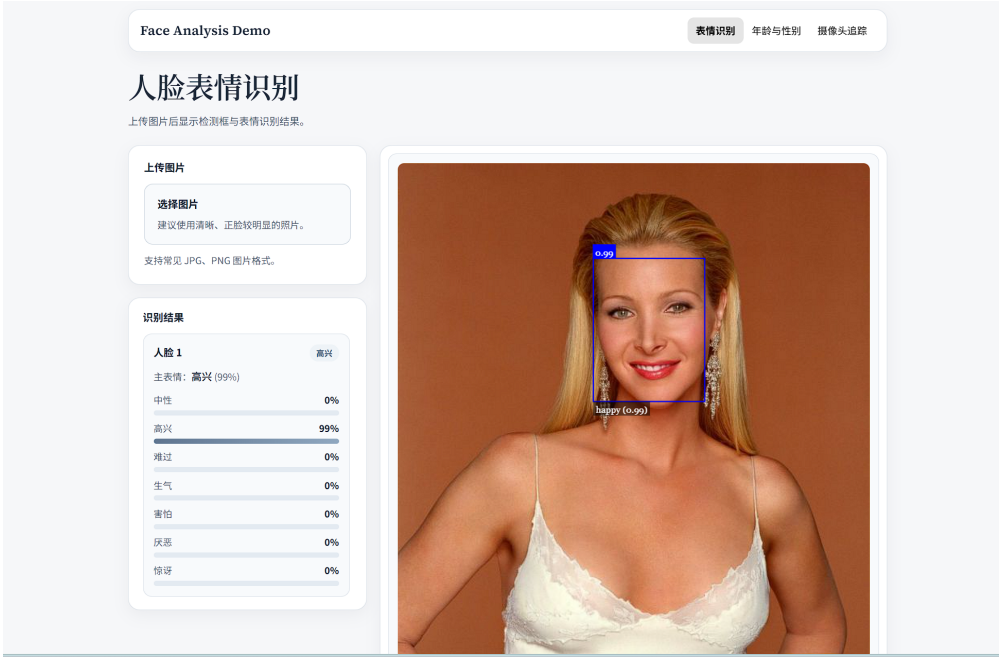


图 1.2 表情识别页面运行截图

1.3 脚本加载与初始化流程

1.3.1 defer 脚本顺序

三个页面都在 `head` 中先引入样式文件，再使用 `defer` 引入脚本。脚本顺序保持为先加载 `dist/face-api.min.js`，再加载当前页面对应的业务脚本。这一顺序是稳定运行的前提，因为业务脚本会直接访问全局对象 `faceapi`。如果业务脚本先执行，

就会出现 faceapiisnotdefined 之类的错误。

defer 的作用是让浏览器在解析 HTML 时并行下载脚本，但等到文档结构解析完成后再按页面中的顺序执行。这样，业务脚本执行时既能访问已经加载好的 faceapi 对象，也能通过 document.getElementById 找到上传控件、结果容器和视频元素。对于静态实验页面来说，这是一种简洁稳定的初始化方式，不需要额外把所有逻辑包在复杂的 DOM ready 回调中。

页面底部还可能引入 jQuery、Popper、Bootstrap 和 MDB 等脚本。这些库主要承担界面组件和样式交互职责，不直接参与模型推理。实验的核心识别链路由 face-api.js 与三个业务脚本完成，其他前端库只改善页面展示体验。

1.3.2 模型加载的并行等待

业务脚本使用 Promise.all 并行加载所需模型。表情识别脚本加载 Tiny Face Detector、68 点关键点、人脸识别、表情识别和 SSD MobileNetV1；年龄性别估计脚本加载 Tiny Face Detector、68 点关键点、人脸识别、年龄性别估计和 SSD MobileNetV1；摄像头脚本加载 Tiny Face Detector、68 点关键点、人脸识别和表情识别模型。只有当这些 Promise 全部完成后，脚本才进入后续的 start 或 startVideo 流程。

这种写法体现了前端模型应用的一个基本原则：在用户交互之前先确保模型可用。否则用户可能已经上传图片，但模型权重还没有加载完成，页面就会进入不确定状态。脚本通过等待模型加载完成后再注册上传监听器或请求摄像头，避免了“输入已经到达但模型尚未准备好”的竞态问题。

不同页面加载的模型略有冗余。例如图片页面加载了 faceRecognitionNet，但当前展示重点不一定使用身份匹配结果。这种设计可能是为了复用脚本模板或保留扩展空间。就实验报告而言，可以把它理解为典型的人脸分析流水线：检测模型先找到人脸，关键点模型定位面部结构，描述子模型提取特征，属性模型输出表情、年龄或性别。

1.4 图片输入处理流程

1.4.1 从本地文件到图像对象

图片类页面通过 input[type=file] 接收用户上传的 JPG、PNG 等常见图片格式。业务脚本监听上传控件的 change 事件。如果用户没有选择文件，脚本直接返回；如果存在文件，页面先渲染“正在识别”的空状态提示，然后清除上一张图片和上一张画布，避免旧结果与新结果叠在一起。

上传文件并不能直接送入模型。脚本调用 faceapi.bufferToImage(imageUpl

oad.files[0])，将本地文件转换为浏览器中的图片对象，再把图片插入展示容器。随后，脚本调用 faceapi.createCanvasFromMedia(image) 创建覆盖画布，使模型标注能够叠加在图片上。这里的 Canvas 是结果可视化的关键：检测框、表情标签、年龄性别标签都不是写在原图上，而是画在覆盖层中。

脚本根据图片的显示尺寸构造 displaySize，再调用 faceapi.matchDimensions(canvas,displaySize) 使画布尺寸与图片一致。这个步骤容易被忽略，但它直接影响标注位置。如果图片显示尺寸与 Canvas 尺寸不一致，模型输出框即使在原始坐标中正确，绘制到页面上也可能发生偏移。

1.4.2 检测结果的尺寸映射

模型推理完成后，脚本调用 faceapi.resizeResults(detections,displaySize) 将检测结果映射到当前显示尺寸。浏览器页面中图片可能受到 CSS 缩放影响，因此模型输出坐标与页面显示坐标不一定完全一致。通过尺寸映射，检测框和属性标签才能准确叠加到用户看到的图片位置上。

图片页面的通用流程可以概括为五步：第一，监听文件上传事件；第二，将文件转换为图片对象并插入页面；第三，创建与图片匹配的 Canvas 覆盖层；第四，调用人脸检测和属性识别接口；第五，将结果映射到显示尺寸，并同时绘制到 Canvas 与渲染到侧边栏。这个流程也说明，浏览器端实验并不是单纯调用一个识别函数，而是包含输入转换、异步等待、尺寸同步、结果排序和界面反馈等多个工程环节。

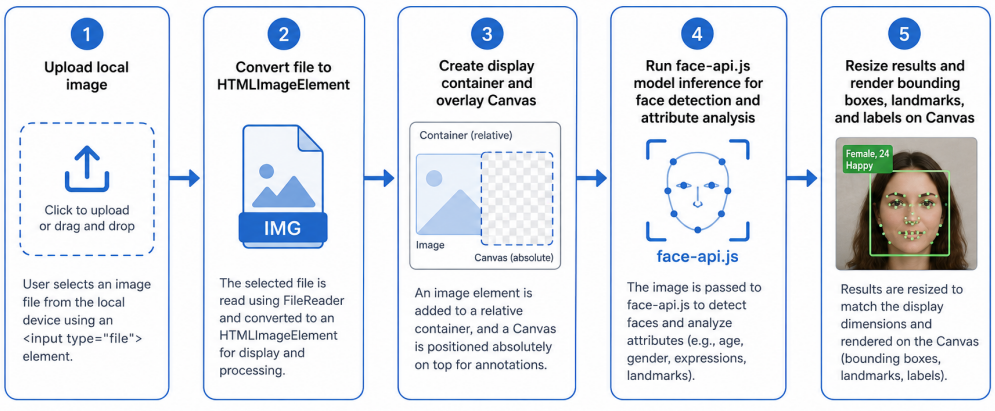


图 1.3 图片输入与画布标注流程

1.5 表情识别模块分析

1.5.1 模型调用链

表情识别由 `dist/expression_script.js` 实现。脚本在图片上传后调用如下链式接口：先执行 `detectAllFaces(image)` 检测所有人脸，再串联 `withFaceLandmarks()` 获取 68 点关键点，接着调用 `withFaceDescriptors()` 提取人脸描述子，最后调用 `withFaceExpressions()` 得到表情分类概率。最终返回的每个人脸结果中包含检测框、关键点、描述子和表情分数。

表情模型输出七类表情概率：`neutral`、`happy`、`sad`、`angry`、`fearful`、`disgusted`、`surprised`。业务脚本将它们映射为中文标签：中性、高兴、难过、生气、害怕、厌恶和惊讶。映射后的标签更适合课程报告和中文界面阅读。

1.5.2 结果排序与概率展示

当图片中存在多张人脸时，脚本并不直接使用模型返回顺序，而是根据检测框的横坐标进行排序，使结果卡片顺序与图片中从左到右的人脸位置一致。这是一个很小但很实用的交互细节。对于多人脸图片，如果侧边栏结果顺序与图像位置不一致，读者很难判断某个概率属于哪张脸。

每张人脸的结果卡片会先显示主表情，即七类概率中分数最高的一项。随后，脚本遍历所有表情类别，用百分比和进度条展示完整概率分布。这样做比只显示一个标签更有解释价值。例如模型可能判断“高兴”为 54%，“中性”为 38%，这说明结果并不是绝对确定，而是在多个类别之间存在竞争。对于课程实验而言，展示概率分布有助于理解模型输出的置信度。

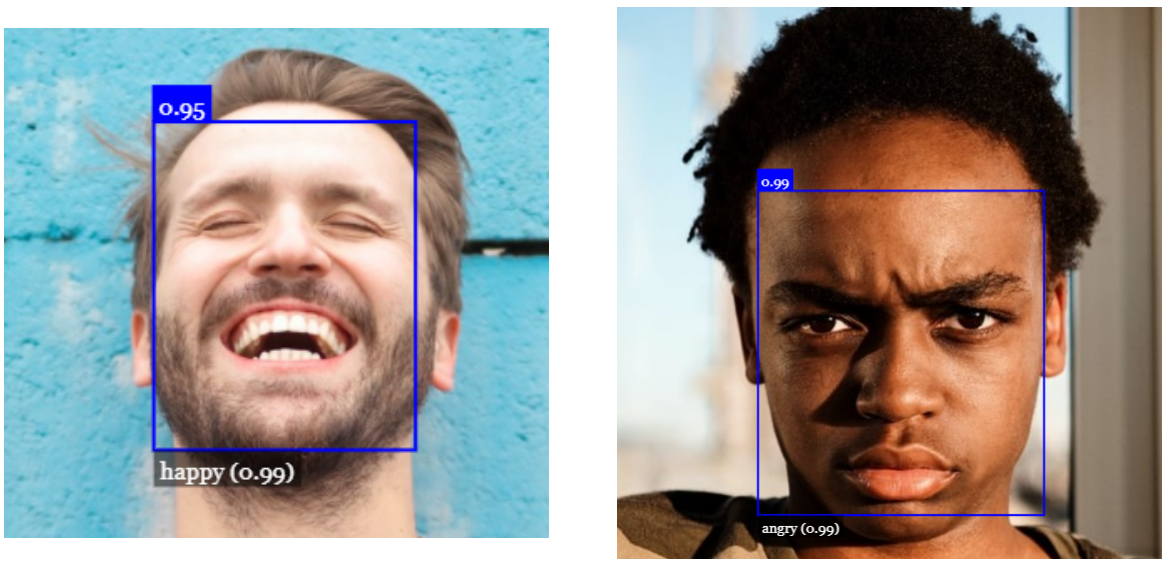


图 1.4 表情识别示例结果

1.5.3 实验现象与局限

表情识别对人脸姿态、遮挡、清晰度和表情幅度比较敏感。正脸、光照均匀、面部无遮挡的图片通常更容易得到稳定结果；侧脸、低分辨率、强背光或面部被手、口罩、头发遮挡时，模型可能无法检测到人脸，或者检测到了人脸但表情概率分布不稳定。表情本身也具有主观性，同一张图片可能同时包含“中性”和“高兴”的特征，因此输出应理解为模型概率估计，而不是人工情绪判断。

从界面反馈看，脚本为“正在识别”和“未检测到人脸”设置了空状态提示。这些状态能降低用户等待时的不确定感，也使页面更接近完整应用，而不是只在控制台打印结果的演示代码。课程报告中记录这些状态，有助于说明实验系统不仅关注模型调用，也关注用户能否理解当前运行阶段。

1.6 年龄与性别估计模块分析

1.6.1 模型调用链与输出字段

年龄与性别估计由 `dist/estimation_script.js` 实现。它与表情识别页面共享类似的图片上传、Canvas 创建和尺寸映射流程，但推理链路末尾换成 `withAgeAndGender()`。模型输出主要包括检测框、估计年龄、性别标签和性别置信度。脚本将 `male` 和 `female` 映射为“男性”和“女性”，并将年龄四舍五入后以“岁”为单位展示。

该模块体现了多任务人脸属性估计的基本思路。人脸检测模型先确定候选区域，关键点模型提供面部结构信息，描述子模型提取可用于表示身份或面部特征的向量，年龄性别模型再在检测到的人脸区域上输出外观属性。虽然脚本使用链式接口写得很短，但背后实际包含多个模型的顺序协作。

1.6.2 Canvas 标注与结果卡片

年龄性别页面使用 `faceapi.draw.drawDetections(canvas,resizedDetections)` 绘制检测框，然后为每张人脸创建 `faceapi.draw.DrawBox`，将年龄和性别字符串作为框标签叠加到图像上。同时，侧边栏结果卡片会展示年龄估计、性别判断和性别置信度。图像标注适合快速定位，侧边栏适合读取更完整的结构化结果。

1.6.3 结果解释与伦理边界

年龄和性别估计结果必须谨慎解释。模型看到的是图像中的外观特征，而不是人的真实身份信息。年龄估计会受到妆容、光照、表情、拍摄角度和图像质量影响；性别估计则涉及更敏感的个人属性，也可能受到训练数据分布和社会偏差影响。因

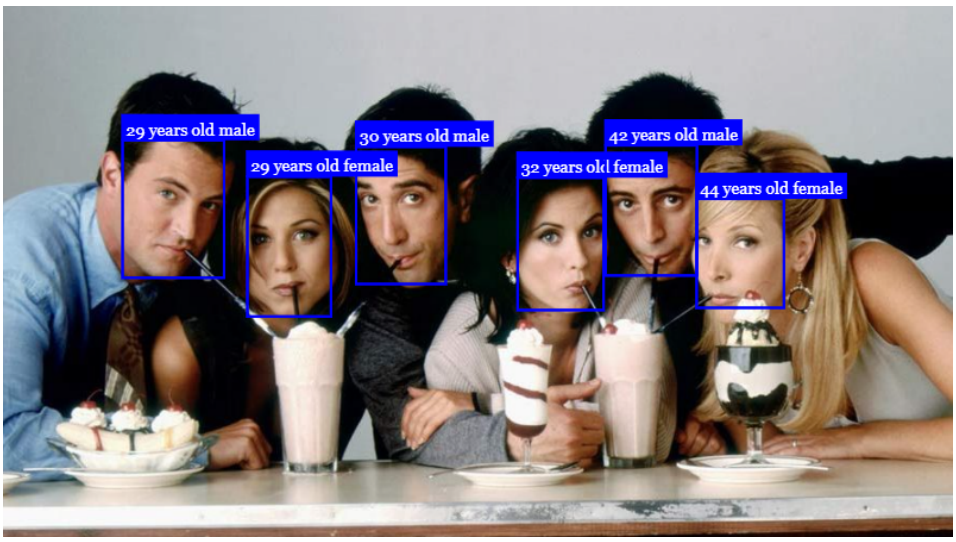


图 1.5　年龄与性别估计示例结果

此，本实验中的结果应称为“模型估计”或“外观属性预测”，不应被表述为真实年龄或真实性别。

在实际应用中，这类属性输出不适合直接用于身份、资格、风险、权限等高影响决策。课程实验可以用它展示模型能力和局限，但报告中应明确说明其不确定性。这样的表述既符合技术事实，也能避免把演示模型包装成可靠判断工具。

年龄与性别估计页面的运行过程可以分为输入准备和结果展示两个阶段。图 1.6 展示了用户在页面中选择待分析图片的状态，此时页面已经提供上传入口和图片显示区域，为后续模型推理准备输入数据。

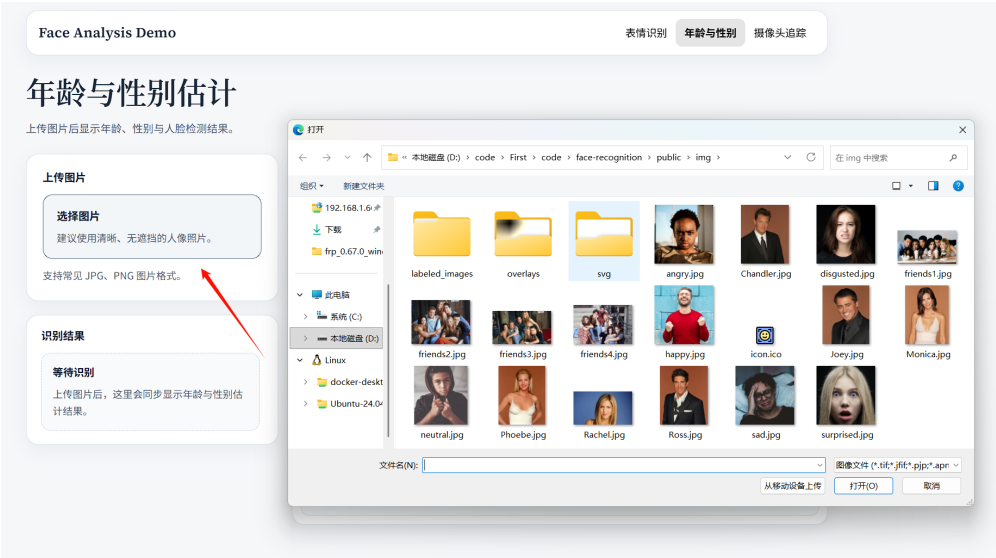


图 1.6　年龄与性别估计页面的图片选择状态

完成图片上传后，脚本调用模型进行人脸检测、年龄估计和性别判断，并将结

果同时绘制在图像区域和结果卡片中。图 1.7 中可以看到，页面在检测框附近叠加了年龄与性别标签，侧边栏则以结构化形式展示估计年龄、性别类别和置信度。



图 1.7　年龄与性别估计页面的结果展示状态

1.7　摄像头实时追踪模块分析

1.7.1　摄像头授权与视频流绑定

摄像头实时追踪由 `dist/webcam_script.js` 实现。页面启动后先显示“等待授权”状态，并检查浏览器是否支持媒体采集接口。脚本优先使用 `navigator.mediaDevices.getUserMedia`，同时保留旧版 `navigator.getUserMedia`、`webkitGetUserMedia` 和 `mozGetUserMedia` 的兼容写法。如果浏览器不支持这些接口，页面会显示“摄像头不可用”；如果用户拒绝授权或设备不可用，页面会显示“访问失败”。

成功获取摄像头视频流后，脚本将 `stream` 赋给 `video.srcObject`。此时浏览器负责把摄像头画面播放到页面中的 `video` 元素。只有当视频触发 `play` 事件后，脚本才创建覆盖画布并启动周期性识别。这种顺序很合理，因为视频尺寸和播放状态尚未确定时，提前创建画布容易导致尺寸不匹配。

1.7.2　周期检测与画布同步

实时追踪的核心是一个 100 ms 间隔的 `setInterval`。每次定时器触发时，脚本读取当前视频帧，调用 `detectAllFaces(video,newfaceapi.TinyFaceDetectorOptions())` 检测人脸，再串联关键点定位和表情识别。推理完成后，脚本将结果映射

到当前显示尺寸，清空上一帧画布内容，并重新绘制检测框、68 点关键点和表情标签。

与图片页面相比，摄像头页面更强调连续性和性能。图片页面只需要处理一次上传图片，摄像头页面则不断处理视频帧。如果每帧都执行复杂模型，低性能设备可能出现卡顿。因此脚本选择 Tiny Face Detector 作为实时检测器，并把识别间隔设置为 100 ms，相当于理论上每秒最多处理约 10 次。这个频率不是视频帧率本身，而是模型推理和标注更新频率。

脚本中的 `syncCanvasSize` 会根据展示容器尺寸同步 Canvas 大小，窗口尺寸变化时也会重新匹配。这个函数解决了实时页面常见的标注错位问题：视频画面可能因为响应式布局变化而改变显示尺寸，如果 Canvas 不跟随更新，检测框就会偏离人脸位置。

1.7.3 结果节流与状态管理

摄像头页面不仅要绘制画布，还要更新侧边栏结果。高频更新 DOM 会带来额外开销，因此脚本设置了 `SIDEBAR_REFRESH_MS=280`，并用 `lastResultsMarkup` 缓存上一次结果。当本次结果与上次相同，或者距离上次渲染时间不足 280 ms 时，脚本会跳过侧边栏更新。这样可以减少重复 DOM 写入，让页面把更多资源留给视频显示和模型推理。

脚本还监听视频的 `pause` 和 `ended` 事件，及时清除检测定时器并更新页面状态。这个细节说明实时推理任务需要生命周期管理：当视频已经暂停，如果定时器仍在后台运行，就会浪费计算资源，也可能造成用户误以为识别还在进行。



图 1.8 摄像头实时追踪运行效果

1.8 实验复现步骤与观察记录

1.8.1 页面访问与资源检查

复现实验时，首先需要确认项目目录中的静态资源完整。三个 HTML 页面必须能够访问 `css/`、`js/`、`dist/`、`models/` 和 `public/` 等目录。若页面打开后只有布局而没有识别能力，通常需要检查 `dist/face-api.min.js` 是否加载成功；若页面能够显示但长时间无法识别，通常需要检查 `models/` 中的权重文件是否存在，以及浏览器控制台是否出现模型加载失败信息。

对于图片类页面，实验观察可以从三个层次记录。第一层是页面结构是否正常，包括导航栏、上传控件、结果区域和图片展示区域是否出现。第二层是模型初始化是否正常，即上传图片后是否能进入“正在识别”状态并最终绘制结果。第三层是识别结果是否合理，包括检测框是否覆盖人脸、侧边栏结果数量是否与图中人脸数量一致、概率或置信度是否以可读形式展示。

对于摄像头页面，复现步骤还需要额外关注浏览器权限。页面必须在浏览器允许摄像头访问的环境下运行；如果浏览器阻止摄像头权限，页面不会进入实时识别阶段。摄像头页面的观察重点包括状态卡片是否从“等待授权”变为“正在连接”或“摄像头已连接”，视频画面是否正常播放，检测框和关键点是否随人脸移动更新，暂停视频后定时器是否停止。

1.8.2 HTML 结构与脚本变量对应

三个页面的 HTML 结构与脚本变量之间存在明确对应关系。表情识别和年龄性别估计页面都包含 `id="upload"` 的文件输入控件，脚本通过 `document.getElementById("upload")` 获取它；右侧结果区域使用 `id="analysis-stage"`，脚本把图片和 Canvas 都插入这个区域；侧边栏结果则分别使用 `id="expression-results"` 和 `id="estimation-results"`。

摄像头页面的关键元素不同。视频元素使用 `id="video"`，脚本把摄像头流绑定到该元素；画布覆盖层插入 `id="tracking-stage"` 对应的容器；摄像头状态写入 `id="webcam-status"`；实时识别结果写入 `id="webcam-results"`。这种 DOM 命名方式让页面结构和业务脚本能够直接对应，便于阅读和调试。

从报告写作角度看，记录这些 DOM 对应关系有两个作用。第一，它能证明实验分析来自实际代码，而不是只描述模型概念。第二，它能解释为什么 HTML 中的元素命名会影响脚本运行。如果某个元素的 `id` 被改动，而脚本仍按旧名称查找，页面就会在初始化阶段出现空引用错误。

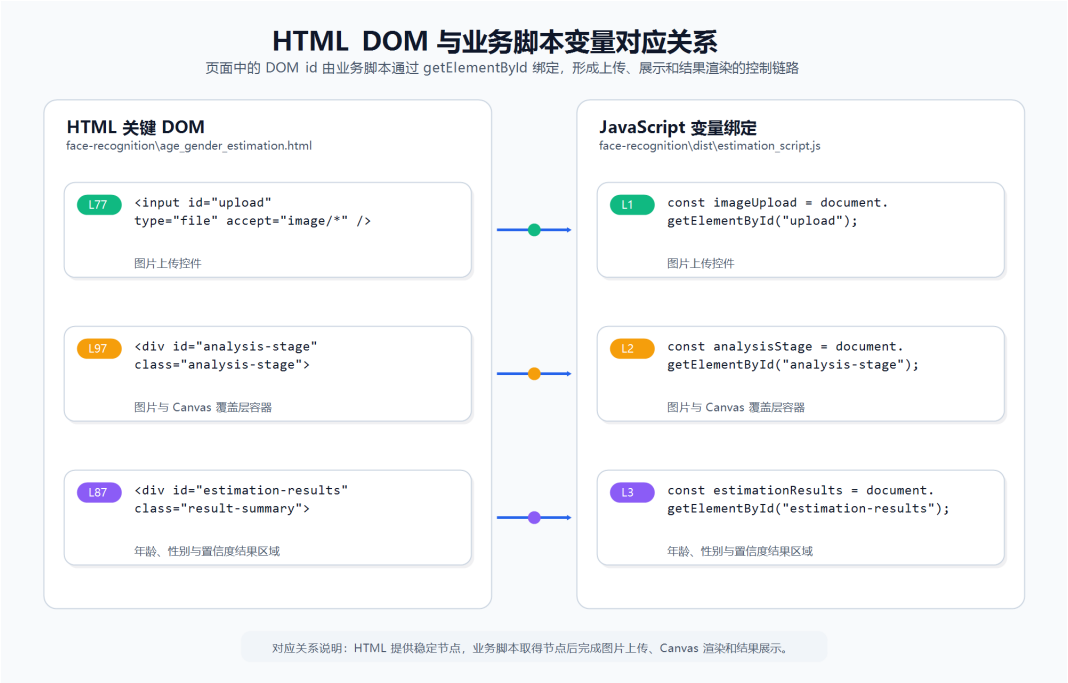


图 1.9　页面 DOM 与业务脚本变量对应关系

1.8.3　建议记录的实验现象

为了让报告更像完整实验，而不是只像代码说明，可以在复现时记录若干具体实验现象。表情识别页面可以分别上传单人正脸、多人合照、侧脸、模糊图片和夸张表情图片，观察检测数量和表情概率变化。年龄性别估计页面可以记录不同年龄段、不同光照和不同姿态图片下的估计波动。摄像头页面可以记录正脸静止、头部转动、多人进入画面和光照变化时的实时标注稳定性。

这些观察不一定需要形成严格统计实验，但应体现输入条件与输出结果之间的关系。例如，当人脸较小或模糊时，检测框可能消失；当表情不明显时，中性概率可能占主导；当摄像头画面快速移动时，关键点可能出现短暂抖动。把这些现象写入报告，可以避免只呈现成功案例，也能体现对模型局限的理解。

1.9　异常场景与边界条件

1.9.1　模型文件加载失败

浏览器端实验最常见的失败场景之一是模型文件加载失败。如果 `MODEL_URL` 指向错误目录，或者 `models/` 中缺少权重文件，`Promise.all` 将无法正常进入后续流程。用户看到的现象可能是页面可以打开，但上传图片后没有反应，或者摄像头页面一直停留在初始状态。此时应优先检查浏览器控制台和网络请求列表。

由于模型加载是异步过程，页面最好提供加载失败提示。当前脚本主要处理正

常加载路径，对于模型加载异常的用户反馈仍有改进空间。课程报告可以把这一点列为后续优化方向：在 `Promise.all` 后补充 `catch` 分支，提示“模型资源加载失败，请检查 models 目录或网络路径”。

1.9.2 输入为空或格式异常

图片页面已经对空文件选择做了简单保护：当 `imageUpload.files.length` 为 0 时，脚本直接返回。由于上传控件使用 `accept="image/*"`，浏览器会尽量限制用户选择非图片文件。但这种限制并不是严格安全边界，实际应用仍应在脚本中处理图片解码失败的异常。

如果图片过大，浏览器端推理可能明显变慢，甚至造成页面短暂无响应。报告中可以说明，课程实验适合使用清晰但尺寸适中的图片。实际应用中可以在前端对图片进行压缩、缩放或限制最大尺寸，再送入模型推理，从而减少计算开销。

1.9.3 摄像头权限与安全上下文

摄像头页面还可能受到浏览器安全策略影响。现代浏览器通常要求摄像头访问发生在安全上下文中，例如 HTTPS 或本机 localhost 环境。若直接以某些文件协议打开页面，摄像头接口可能不可用。脚本虽然兼容新旧 `getUserMedia` 接口，但无法绕过浏览器权限策略。

用户拒绝授权、设备被其他程序占用、摄像头不存在或浏览器不支持媒体采集接口，都会导致实时追踪失败。当前脚本会显示“访问失败”或“不支持”等状态，这对于实验页面已经比较友好。后续若扩展为正式应用，还可以增加更具体的错误分类，例如权限拒绝、设备未找到、设备忙碌和安全上下文不满足。

1.10 评价指标与实验扩展

1.10.1 检测类指标

如果需要把本实验从功能展示扩展为定量分析，可以首先设计检测类指标。对于人脸检测，可统计测试图片总数、真实人脸总数、成功检测人脸数、漏检数和误检数。检测成功率可以粗略表示为成功检测人脸数与真实人脸总数之比。对于多人脸图片，还可以记录检测框位置是否与真实人脸区域基本重合。

课程报告不一定需要构建大规模数据集，但可以选取若干代表性样例：正脸单人、多人合照、侧脸、强光、弱光、遮挡、低清晰度等。通过这些样例记录模型表现，可以让实验结论更有依据。例如，报告可以说明“在正脸清晰图片上识别较稳定，在遮挡和侧脸场景中容易漏检”。

1.10.2 属性估计指标

对于表情识别，评价可以观察主表情是否符合人工直观判断，以及概率分布是否集中。对于年龄估计，可以记录模型估计值与人工标注年龄的大致偏差，但要注意隐私和数据来源。对于性别估计，不建议把结果作为强标签评价，更适合讨论模型输出的不确定性和潜在偏差。

属性估计的结果不应只追求“准确率”一个数字。表情、年龄和性别都涉及主观性、社会属性和数据偏差。更合理的写法是结合置信度、输入质量和适用边界进行分析。尤其在课程报告中，应把这些输出理解为模型能力展示，而不是可靠身份判断。

1.10.3 实时性能指标

摄像头页面可以记录实时性能指标，例如模型更新间隔、页面体感延迟、检测框抖动情况和设备资源占用。当前脚本的检测间隔设置为 100 ms，但实际推理耗时可能超过 100 ms；当推理时间较长时，页面无法真正达到每秒 10 次稳定更新。报告中可以说明，定时器间隔是目标频率，实际频率还取决于设备性能和模型耗时。

如果后续要优化实时性能，可以考虑降低视频显示分辨率、减少绘制内容、只绘制检测框不绘制关键点、延长检测间隔，或者使用更轻量的检测参数。优化方向需要在速度、稳定性和结果丰富度之间取舍。

1.11 综合实验结果与分析

1.11.1 功能完整性

从功能完整性看，该项目完成了三个典型的人脸分析任务。表情识别页面能够在上传图片后检测多张人脸，并输出每张人脸的主表情和七类表情概率；年龄与性别估计页面能够输出检测框、估计年龄、性别标签和置信度；摄像头追踪页面能够在获得权限后持续处理视频帧，并实时叠加检测框、关键点和表情结果。

从工程实现看，项目具有较好的演示性。页面入口清楚，模型资源保存在本地，业务脚本集中在 `dist/`，示例图片和人物图片保存在 `public/`。这种组织方式方便教师讲解，也方便学生对照代码与报告复现实验。相比只展示最终效果，报告中分析文件结构和脚本流程更能体现课程实验的完整性。

1.11.2 性能影响因素

浏览器端推理的性能主要受三个因素影响。第一是模型加载时间。首次打开页面时，浏览器需要读取多个模型权重文件，模型越多，初始等待越明显。第二是单

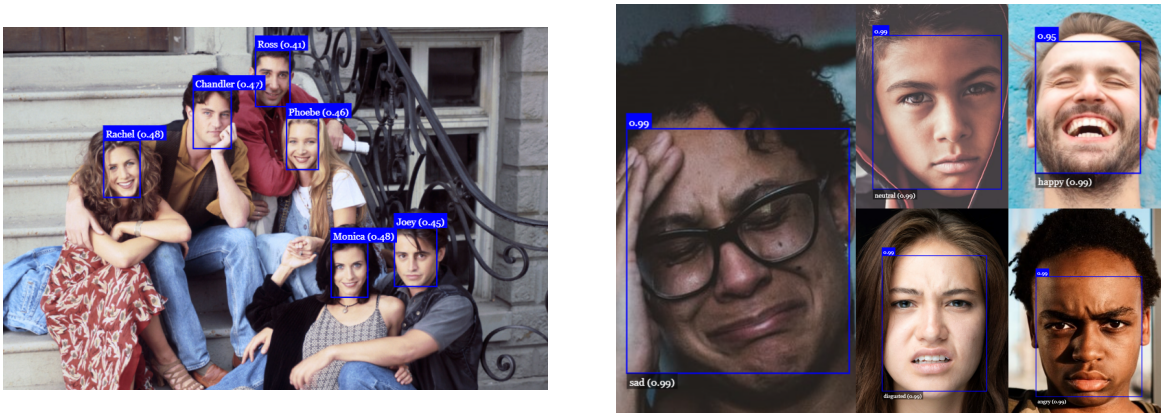


图 1.10　人脸检测与综合演示效果

次推理时间。图片越大、人脸越多、模型越复杂，推理时间越长。第三是页面渲染成本。Canvas 绘制和 DOM 更新虽然不属于神经网络计算，但在实时任务中也会影响流畅度。

摄像头页面还受到设备摄像头分辨率、CPU/GPU 能力、浏览器实现和当前系统负载影响。脚本通过 Tiny Face Detector、100 ms 检测间隔和侧边栏节流来控制开销，但并不能保证所有设备都有相同效果。如果输入视频分辨率较高或设备性能较弱，实时标注可能出现延迟、掉帧或结果抖动。

1.11.3　可靠性与数据质量

人脸分析结果对输入质量依赖明显。正脸、光照均匀、面部无遮挡的图像通常更容易得到稳定结果；侧脸、模糊、强背光、表情幅度小、多人人脸重叠或遮挡严重的图像可能导致漏检、误检或属性估计偏差。对于摄像头场景，用户移动、光照变化和镜头自动曝光也会影响每一帧结果。

因此，实验报告不应只写“系统能够识别”，还应写清它在什么条件下更容易识别、在什么条件下可能失败。这样的分析能体现对模型应用边界的理解。尤其是年龄、性别和表情等属性识别，输出本身带有概率性和不确定性，不适合脱离实验语境做确定判断。

1.12　隐私、安全与改进方向

摄像头功能涉及人脸和视频流等敏感信息。当前项目在浏览器端本地处理数据，没有主动把视频流上传到服务器，这是浏览器端部署的一个隐私优势。但是，只要页面请求摄像头权限，就应该让用户理解授权含义。课程报告中应说明：实验只用于本地演示，不应保存、传播或公开未经同意的人脸图像。

从工程改进角度看，项目还可以继续优化。第一，可以抽取公共逻辑，例如模

型加载、空状态渲染、检测结果排序、Canvas 尺寸同步等，减少三个脚本之间的重复。第二，可以加入更明确的加载进度和错误提示，例如模型文件缺失、图片格式不支持、摄像头权限拒绝等。第三，可以构造定量测试集，统计不同光照、姿态、遮挡和多人脸场景下的检测成功率、属性稳定性和实时帧率。

从实验写作角度看，后续还可以补充更多真实运行截图。图片页面截图能证明上传和标注流程正常，摄像头截图能证明实时追踪有效，目录结构截图能证明模型和资源均来自本地仓库。相比只放模型示例图，真实运行截图更能体现课程项目的复现过程。

图片类人脸分析页面如图 1.11 所示。表情识别页面以图片上传为入口，右侧用于显示待分析图片和 Canvas 标注结果，左侧展示表情类别和概率分布；年龄与性别估计页面采用相似的上传和展示结构，但侧边栏结果重点变为估计年龄、性别类别和置信度，体现了同一套前端推理流程在不同属性任务中的复用。



（a）表情识别页面　　（b）年龄与性别估计页面

图 1.11　图片类人脸分析页面运行界面

摄像头追踪页面如图 1.12 所示。该页面以实时视频流作为输入，在授权摄像头后持续更新检测框、关键点和表情结果，更能体现浏览器端模型推理在连续输入场景下的交互特点。

1.13　本章小结

本章围绕 `face-recognition` 项目分析了浏览器端人脸分析实验。实验展示了静态网页如何利用 `face-api.js` 加载本地预训练模型，并在图片和摄像头视频流两类输入上完成人脸检测、关键点定位、表情识别以及年龄性别估计。章节重点放在前端推理阶段，包括页面结构、模型加载、输入转换、Canvas 绘制、结果渲染和状态管理。

通过三个模块的对比可以看出，浏览器端深度学习推理适合课程演示、交互原型和轻量级体验场景。它降低了后端部署成本，也减少了敏感图像上传的需求。但

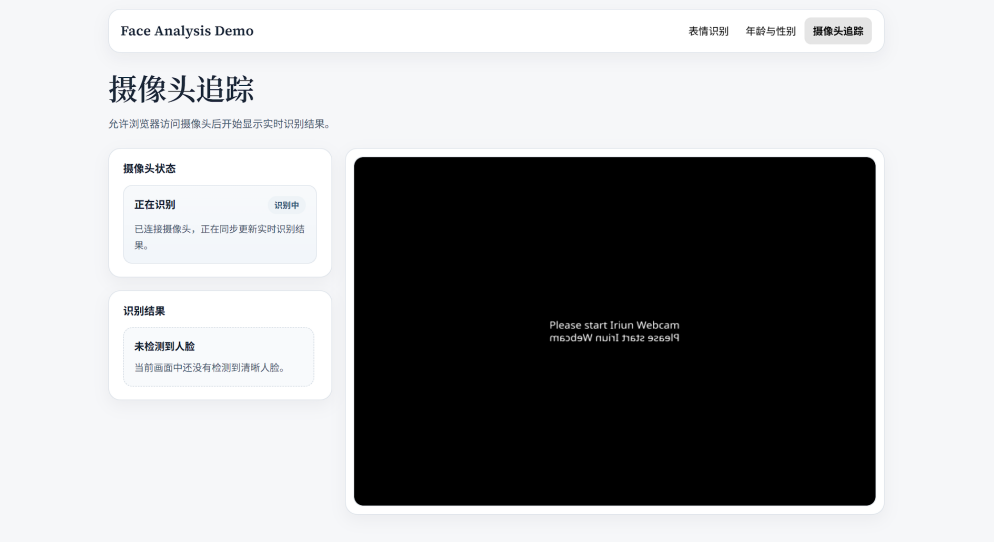


图 1.12　摄像头实时追踪页面运行界面

它仍然受到设备性能、浏览器环境、输入质量和模型偏差影响。因此，在分析实验结果时，既要关注页面能否完成检测和属性输出，也要结合输入质量、运行环境和模型适用范围理解结果的可靠性。

参考文献

[1] Vincent Muhler. face-api.js: Javascript api for face detection and face recognition in the browser and node.js with tensorflow.js. https://github.com/justadudewhohacks/face-api.js, 2018. Accessed: 2026-05-20.

[2] Daniel Smilkov, Nikhil Thorat, Yannick Assogba, Charles Nicholson, Nick Kreeger, Ping Yu, Shanqing Cai, Eric Nielsen, David Soegel, Stan Bileschi, Michael Terry, Ann Yuan, Kangyi Zhang, Sandeep Gupta, Sarah Sirajuddin, D. Sculley, Rajat Monga, Greg Corrado, Fernanda Viegas, and Martin M. Wattenberg. Tensorflow.js: Machine learning for the web and beyond. In *Proceedings of Machine Learning and Systems*, volume 1, pages 309–321, 2019. https://research.google/pubs/tensorflowjs-machine-learning-for-the-web-and-beyond/.

[3] Davis E. King. Dlib-ml: A machine learning toolkit. *Journal of Machine Learning Research*, 10(60):1755–1758, 2009. https://jmlr.csail.mit.edu/papers/v10/king09a.html.

[4] Davis E. King. dlib c++ library. https://github.com/davisking/dlib. Accessed: 2026-05-20.

[5] Florian Schroff, Dmitry Kalenichenko, and James Philbin. Facenet: A unified embedding for face recognition and clustering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 815–823, 2015. https://openaccess.thecvf.com/content_cvpr_2015/html/Schroff_FaceNet_A_Unified_2015_CVPR_paper.html.

[6] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. doi: 10.1109/CVPR.2016.90; https://arxiv.org/abs/1512.03385.

[7] World Wide Web Consortium. Media capture and streams. https://www.w3.org/TR/mediacapture-streams/, 2025. W3C Candidate Recommendation Draft, accessed: 2026-05-20.

[8] Bootstrap Team. Bootstrap documentation. https://getbootstrap.com/docs/5.3/getting-started/introduction/, 2026. Accessed: 2026-05-20.

[9] Unity Technologies. Rendering in the universal render pipeline. https://docs.unity3d.com/6000.0/Documentation/Manual/urp/rendering-in-universalrp.html, 2026. Unity Manual, accessed: 2026-05-20.

[10] Unity Technologies. Upgrade material assets to urp or hdrp. https://docs.unity3d.com/6000.0/Documentation/Manual/upgrade-material.html, 2026. Unity Manual, accessed: 2026-05-20.

[11] Unity Technologies. Locomotion overview: Xr interaction toolkit. https://docs.unity3d.com/Packages/com.unity.xr.interaction.toolkit@3.1/manual/locomotion.html, 2025. Unity XR Interaction Toolkit Manual, accessed: 2026-05-20.

[12] Khronos OpenXR Working Group. The openxr specification. https://registry.khronos.org/OpenXR/, 2026. Accessed: 2026-05-20.

[13] TA TriForge Assets. Fantasy forest environment - free demo. https://assetstore.unity.com/packages/3d/environments/fantasy/fantasy-forest-environment-free-demo-35361, 2019. Unity Asset Store, accessed: 2026-05-20.

[14] Poly Ronin. Low poly pine forest. https://assetstore.unity.com/packages/3d/environments/low-poly-pine-forest-312113, 2025. Unity Asset Store, accessed: 2026-05-20.

[15] ithappy. Animals free - animated low poly 3d models. https://assetstore.unity.com/packages/3d/characters/animals/animals-free-animated-low-poly-3d-models-260727, 2025. Unity Asset Store, accessed: 2026-05-20.

附录　实验关键代码

本附录节选报告实验中与功能实现直接相关的关键代码，便于读者对照正文中的流程分析查看具体实现。附录不完整粘贴压缩后的第三方库源码，例如 face-api.min.js，而是保留页面入口、展示容器、模型加载、图片推理、身份匹配、结果渲染和摄像头实时追踪等关键片段。同时，附录也摘录 dlib 人脸识别训练示例和 Unity 森林 VR 场景中的 Shader、移动系统代码，用于说明模型训练思想和场景交互实现。

页面入口与展示容器

图片表情识别页面在文档头部延迟加载 face-api.min.js 和当前页面的业务脚本。主体区域左侧提供图片上传控件和识别结果容器，右侧的 analysis-stage 用于插入用户上传的图片和覆盖绘制结果的画布。

这里保留脚本标签的原因是，代码引入顺序直接影响页面能否运行。face-api.min.js 必须先于业务脚本加载，业务脚本才能访问 faceapi 对象；defer 则保证脚本在 DOM 解析完成后按顺序执行，使业务脚本能够拿到上传控件、展示区域等真实节点。

```
21    <script defer src="dist/face-api.min.js"></script>
22    <script defer src="dist/expression_script.js"></script>
```

```
75              <section class="panel control-panel">
76                <h2 class="panel-heading"> 上传图片 </h2>
77                <input id="upload" type="file" accept="image/*" />
78                <label for="upload" class="upload-dropzone">
79                  <span class="upload-label"> 选择图片 </span>
80                  <span class="upload-hint"> 建议使用清晰、正脸较明显的照片。</span>
81                </label>
82                <p class="panel-note"> 支持常见 JPG、PNG 图片格式。</p>
83              </section>

84              <section class="panel info-panel">
85                <h2 class="panel-heading"> 识别结果 </h2>
86                <div id="expression-results" class="result-summary">
87                  <div class="result-empty">
88                    <strong> 等待识别 </strong>
89                    <span> 上传图片后，这里会同步显示表情识别结果副本。</span>
90                  </div>
91                </div>
92              </section>
93            </aside>

94            <section class="panel result-panel">
95              <div id="analysis-stage" class="analysis-stage">
96                <div class="stage-empty" data-empty-state>
97                  <strong> 等待图片 </strong>
98                  <span> 上传后在这里显示表情识别结果。</span>
99                </div>
```

年龄与性别估计页面的入口结构与表情识别页面基本一致，只是业务脚本替换

为 dist/estimation_script.js，结果容器也变为 estimation-results。这种“页面结构相同、业务脚本不同”的组织方式有利于复用样式和交互习惯。

```
21    <script defer src="dist/face-api.min.js"></script>
22    <script defer src="dist/estimation_script.js"></script>
```

```
75              <section class="panel control-panel">
76                <h2 class="panel-heading"> 上传图片 </h2>
77                <input id="upload" type="file" accept="image/*" />
78                <label for="upload" class="upload-dropzone">
79                  <span class="upload-label"> 选择图片 </span>
80                  <span class="upload-hint"> 建议使用清晰、无遮挡的人像照片。</span>
81                </label>
82                <p class="panel-note"> 支持常见 JPG、PNG 图片格式。</p>
83              </section>

84              <section class="panel info-panel">
85                <h2 class="panel-heading"> 识别结果 </h2>
86                <div id="estimation-results" class="result-summary">
87                  <div class="result-empty">
88                    <strong> 等待识别 </strong>
89                    <span> 上传图片后，这里会同步显示年龄与性别估计结果。</span>
90                  </div>
91                </div>
92              </section>
93            </aside>

94            <section class="panel result-panel">
95              <div id="analysis-stage" class="analysis-stage">
96                <div class="stage-empty" data-empty-state>
97                  <strong> 等待图片 </strong>
98                  <span> 上传后在这里显示年龄与性别估计结果。</span>
```

摄像头实时追踪页面同样通过脚本标签加载模型库和业务逻辑。页面中的追踪区域 tracking-stage 用来叠加视频与检测画布。

video 元素负责承接浏览器摄像头视频流，并为后续逐帧识别提供输入。

```
21    <script defer src="dist/face-api.min.js"></script>
22    <script defer src="dist/webcam_script.js"></script>
```

```
101           <section class="panel tracking-panel">
102             <div id="tracking-stage" class="tracking-stage">
103               <div class="stage-empty" data-empty-state>
104                 <strong> 等待摄像头权限 </strong>
105                 <span> 授权后在这里显示实时画面与识别结果。</span>
106               </div>
107               <video
108                 id="video"
109                 width="1280"
110                 height="720"
111                 autoplay
112                 muted
113                 playsinline
114               ></video>
115             </div>
```

图片表情识别脚本

表情识别脚本首先取得页面中的上传控件、展示区域和结果区域，并定义本地模型路径。随后通过 `Promise.all` 并行加载检测、关键点、人脸描述子、表情识别和 SSD MobileNetV1 模型，全部模型加载完成后才进入 `start` 流程。

```
const imageUpload = document.getElementById("upload");
const analysisStage = document.getElementById("analysis-stage");
const expressionResults = document.getElementById("expression-results");
const MODEL_URL = "./models";

const EXPRESSION_LABELS = {
  neutral: " 中性",
  happy: " 高兴",
  sad: " 难过",
  angry: " 生气",
  fearful: " 害怕",
  disgusted: " 厌恶",
  surprised: " 惊讶",
};
const EXPRESSION_ORDER = Object.keys(EXPRESSION_LABELS);

Promise.all([
  faceapi.nets.tinyFaceDetector.loadFromUri(MODEL_URL),
  faceapi.nets.faceLandmark68Net.loadFromUri(MODEL_URL),
  faceapi.nets.faceRecognitionNet.loadFromUri(MODEL_URL),
  faceapi.nets.faceExpressionNet.loadFromUri(MODEL_URL),
  faceapi.nets.ssdMobilenetv1.loadFromUri(MODEL_URL),
]).then(start);
```

这一段代码体现了两个关键点。第一，`MODEL_URL` 指向 `./models`，说明权重从项目本地目录加载，不依赖远程网络。第二，模型加载使用 `Promise.all`，所有异步加载任务都完成后才调用 `start`。如果缺少这一步等待，用户上传图片时模型可能尚未准备好，推理调用会失败。

为了让模型输出更容易阅读，脚本将 `face-api.js` 返回的英文表情标签映射为中文标签，并按检测框横坐标排序多张人脸。结果渲染函数会选出概率最高的主表情，同时把七类表情概率渲染为进度条，便于观察模型对不同表情类别的置信度分布。

```
function getSortedDetections(detections) {
  return [...detections].sort(
    (first, second) => first.detection.box.x - second.detection.box.x
  );
}

function getTopExpression(expressions) {
  return EXPRESSION_ORDER.reduce(
    (currentTop, key) => {
      const score = expressions[key] || 0;
      return score > currentTop.score ? { key, score } : currentTop;
    },
```

```
    { key: EXPRESSION_ORDER[0], score: expressions[EXPRESSION_ORDER[0]] || 0 }
  );
}

function renderExpressionResults(detections) {
  const sortedDetections = getSortedDetections(detections);

  if (!sortedDetections.length) {
    renderEmptyState(
      expressionResults,
      " 未检测到人脸",
      " 请尝试上传清晰、正脸更明显的照片。"
    );
    return;
  }

  expressionResults.innerHTML = sortedDetections
    .map((detection, index) => {
      const topExpression = getTopExpression(detection.expressions);
      const metrics = EXPRESSION_ORDER.map((key) => {
        const score = detection.expressions[key] || 0;

        return `
          <li class="result-metric">
            <div class="result-metric__row">
              <span class="result-metric__label">${EXPRESSION_LABELS[key]}</span>
              <span class="result-metric__value">${formatPercent(score)}</span>
            </div>
            <div class="result-bar"><span style="width: ${Math.round(
              score * 100
            )}%"></span></div>
          </li>
        `;
      }).join("");

      return `
        <article class="result-card">
          <div class="result-card__header">
            <span class="result-card__title"> 人脸 ${index + 1}</span>
            <span class="status-pill">${EXPRESSION_LABELS[topExpression.key]}</span>
          </div>
          <p class="result-card__highlight">
            主表情: <strong>${EXPRESSION_LABELS[topExpression.key]}</strong>
            (${formatPercent(topExpression.score)})
          </p>
          <ul class="result-list">${metrics}</ul>
        </article>
      `;
    })
    .join("");
}
```

图片上传监听器是该模块的核心入口。用户选择文件后，脚本先清理旧图片和旧画布，再将文件转换为浏览器图片对象，创建覆盖画布并匹配尺寸。随后，脚本依次执行人脸检测、关键点定位、人脸描述子提取和表情识别，最后将结果缩放到展示尺寸并绘制检测框和表情标签。

```
async function start() {
```

```
  const container = document.createElement("div");
  container.className = "analysis-stage__media";
  analysisStage.append(container);

  let image;
  let canvas;

  imageUpload.addEventListener("change", async () => {
    if (!imageUpload.files.length) {
      return;
    }

    renderEmptyState(expressionResults, " 正在识别", " 图片分析中，请稍候。");

    if (image) {
      image.remove();
    }
    if (canvas) {
      canvas.remove();
    }

    image = await faceapi.bufferToImage(imageUpload.files[0]);
    container.append(image);

    canvas = faceapi.createCanvasFromMedia(image);
    container.append(canvas);

    const displaySize = { width: image.width, height: image.height };
    faceapi.matchDimensions(canvas, displaySize);

    const detections = await faceapi
      .detectAllFaces(image)
      .withFaceLandmarks()
      .withFaceDescriptors()
      .withFaceExpressions();

    const resizedDetections = faceapi.resizeResults(detections, displaySize);
    faceapi.draw.drawDetections(canvas, resizedDetections);
    faceapi.draw.drawFaceExpressions(canvas, resizedDetections);
    renderExpressionResults(resizedDetections);

    analysisStage.classList.add("stage-ready");
  });
}
```

其中 `faceapi.bufferToImage` 负责把本地文件对象转换成浏览器可绘制的图片元素，`createCanvasFromMedia` 创建与图片对应的覆盖画布，`matchDimensions` 保证画布坐标系与图片显示尺寸一致。链式调用 `detectAllFaces(image).withFaceLandmarks().withFaceDescriptors().withFaceExpressions()` 表示先检测所有人脸，再补充关键点、人脸描述子和表情结果。最后的 `resizeResults` 用于把模型输出坐标映射回展示尺寸。

年龄与性别估计脚本

年龄与性别估计脚本与表情识别脚本结构相近，但加载的任务模型不同。它除了检测、关键点和描述子模型外，还加载 ageGenderNet，用于输出估计年龄、性别标签和性别分类置信度。

```
const imageUpload = document.getElementById("upload");
const analysisStage = document.getElementById("analysis-stage");
const estimationResults = document.getElementById("estimation-results");
const MODEL_URL = "./models";

const GENDER_LABELS = {
  male: " 男性",
  female: " 女性",
};

Promise.all([
  faceapi.nets.tinyFaceDetector.loadFromUri(MODEL_URL),
  faceapi.nets.faceLandmark68Net.loadFromUri(MODEL_URL),
  faceapi.nets.faceRecognitionNet.loadFromUri(MODEL_URL),
  faceapi.nets.ageGenderNet.loadFromUri(MODEL_URL),
  faceapi.nets.ssdMobilenetv1.loadFromUri(MODEL_URL),
]).then(start);
```

这一模块同样使用本地 models 目录。与表情识别相比，它把任务网络从 faceExpressionNet 换成 ageGenderNet。这说明页面的基础流程可以复用：只要输入仍然是人脸区域，后续可以在同一检测结果上接入不同属性估计网络。

结果渲染部分把模型返回的 male 和 female 映射为中文展示文本，并将年龄估计值四舍五入后写入结果卡片。性别置信度继续用百分比和进度条表示，使侧边栏结果能够与右侧检测框形成对照。

```
function renderEstimationResults(detections) {
  const sortedDetections = getSortedDetections(detections);

  if (!sortedDetections.length) {
    renderEmptyState(
      estimationResults,
      " 未检测到人脸",
      " 请尝试上传清晰、无遮挡的人像照片。"
    );
    return;
  }

  estimationResults.innerHTML = sortedDetections
    .map((detection, index) => {
      const gender = detection.gender || "male";
      const genderLabel = GENDER_LABELS[gender] || gender;
      const confidence = detection.genderProbability || 0;

      return `
        <article class="result-card">
          <div class="result-card__header">
            <span class="result-card__title"> 人脸 ${index + 1}</span>
```

```
            <span class="status-pill">${genderLabel}</span>
          </div>
          <div class="result-detail-grid">
            <div class="result-detail">
              <span class="result-detail__label"> 年龄估计 </span>
              <span class="result-detail__value">${Math.round(
                detection.age
              )} 岁 </span>
            </div>
            <div class="result-detail">
              <span class="result-detail__label"> 性别判断 </span>
              <span class="result-detail__value">${genderLabel}</span>
            </div>
          </div>
          <ul class="result-list">
            <li class="result-metric">
              <div class="result-metric__row">
                <span class="result-metric__label"> 性别置信度 </span>
                <span class="result-metric__value">${formatPercent(
                  confidence
                )}</span>
              </div>
              <div class="result-bar"><span style="width: ${Math.round(
                confidence * 100
              )}%"></span></div>
            </li>
          </ul>
        </article>
      `;
    })
    .join("");
}
```

图片推理流程仍然从上传事件开始。与表情识别不同的是，链式调用最后使用 `withAgeAndGender`，并在画布上额外通过 `faceapi.draw.DrawBox` 绘制包含年龄和性别的检测框标签。

```
async function start() {
  const container = document.createElement("div");
  container.className = "analysis-stage__media";
  analysisStage.append(container);

  let image;
  let canvas;

  imageUpload.addEventListener("change", async () => {
    if (!imageUpload.files.length) {
      return;
    }

    renderEmptyState(estimationResults, " 正在识别", " 图片分析中，请稍候。");

    if (image) {
      image.remove();
    }
    if (canvas) {
      canvas.remove();
    }
```

```
    image = await faceapi.bufferToImage(imageUpload.files[0]);
    container.append(image);

    canvas = faceapi.createCanvasFromMedia(image);
    container.append(canvas);

    const displaySize = { width: image.width, height: image.height };
    faceapi.matchDimensions(canvas, displaySize);

    const detections = await faceapi
      .detectAllFaces(image)
      .withFaceLandmarks()
      .withFaceDescriptors()
      .withAgeAndGender();

    const resizedDetections = faceapi.resizeResults(detections, displaySize);
    faceapi.draw.drawDetections(canvas, resizedDetections);

    resizedDetections.forEach((detection, index) => {
      const box = resizedDetections[index].detection.box;
      const drawBox = new faceapi.draw.DrawBox(box, {
        label: `${Math.round(detection.age)} years old ${detection.gender}`,
      });
      drawBox.draw(canvas);
    });

    renderEstimationResults(resizedDetections);

    analysisStage.classList.add("stage-ready");
  });
}
```

从代码结构看，年龄与性别估计和表情识别最大的差异只在两处：一是模型加载列表不同，二是链式推理末端不同。其余的文件读取、图片插入、画布创建、尺寸匹配、检测结果缩放等流程完全相似。这也说明前端人脸分析页面适合按“基础检测流程 + 任务网络”的方式扩展。

摄像头实时追踪脚本

摄像头脚本在页面初始化时保存视频元素、追踪区域、状态区和结果区引用，并维护覆盖画布、定时器和结果刷新时间等状态。模型加载完成后进入 `startVideo`，为后续实时识别做准备。

```
const video = document.getElementById("video");
const trackingStage = document.getElementById("tracking-stage");
const webcamStatus = document.getElementById("webcam-status");
const webcamResults = document.getElementById("webcam-results");
const MODEL_URL = "./models";
let canvas;
let detectionInterval;
let lastResultsMarkup = "";
let lastResultsRenderAt = 0;
```

```
const EXPRESSION_LABELS = {
  neutral: " 中性",
  happy: " 高兴",
  sad: " 难过",
  angry: " 生气",
  fearful: " 害怕",
  disgusted: " 厌恶",
  surprised: " 惊讶",
};
const EXPRESSION_ORDER = Object.keys(EXPRESSION_LABELS);
const SIDEBAR_REFRESH_MS = 280;

Promise.all([
  faceapi.nets.tinyFaceDetector.loadFromUri(MODEL_URL),
  faceapi.nets.faceLandmark68Net.loadFromUri(MODEL_URL),
  faceapi.nets.faceRecognitionNet.loadFromUri(MODEL_URL),
  faceapi.nets.faceExpressionNet.loadFromUri(MODEL_URL),
]).then(startVideo);

function getDisplaySize() {
  return {
    width: Math.max(trackingStage.clientWidth, 1),
    height: Math.max(trackingStage.clientHeight, 1),
  };
}

function syncCanvasSize() {
  if (!canvas) {
    return getDisplaySize();
  }

  const displaySize = getDisplaySize();

  if (canvas.width !== displaySize.width || canvas.height !== displaySize.height) {
    faceapi.matchDimensions(canvas, displaySize);
  }

  return displaySize;
}
```

与图片页面不同，摄像头页面需要维护跨帧状态。`canvas` 保存覆盖画布，`detectionInterval` 保存定时器编号，`lastResultsMarkup` 和 `lastResultsRenderAt` 用于控制侧边栏刷新频率。这些变量保证页面在连续视频输入下不会不断创建新画布或过度更新 DOM。

实时页面需要频繁更新侧边栏，因此脚本把结果 HTML 的生成和渲染拆开处理。渲染函数会比较本次结果与上次结果，并使用 `SIDEBAR_REFRESH_MS` 控制刷新间隔，避免每 100 ms 都重复写入 DOM。

```
function getTopExpression(expressions) {
  return EXPRESSION_ORDER.reduce(
    (currentTop, key) => {
      const score = expressions[key] || 0;
      return score > currentTop.score ? { key, score } : currentTop;
    },
    { key: EXPRESSION_ORDER[0], score: expressions[EXPRESSION_ORDER[0]] || 0 }
```

```
  );
}

function buildResultsMarkup(detections) {
  const sortedDetections = getSortedDetections(detections);

  if (!sortedDetections.length) {
    return `
      <div class="result-empty">
        <strong> 未检测到人脸 </strong>
        <span> 当前画面中还没有检测到清晰人脸。</span>
      </div>
    `;
  }

  return sortedDetections
    .map((detection, index) => {
      const topExpression = getTopExpression(detection.expressions);
      const metrics = EXPRESSION_ORDER.map((key) => {
        const score = detection.expressions[key] || 0;

        return `
          <li class="result-metric">
            <div class="result-metric__row">
              <span class="result-metric__label">${EXPRESSION_LABELS[key]}</span>
              <span class="result-metric__value">${formatPercent(score)}</span>
            </div>
            <div class="result-bar"><span style="width: ${Math.round(
              score * 100
            )}%"></span></div>
          </li>
        `;
      }).join("");

      return `
        <article class="result-card">
          <div class="result-card__header">
            <span class="result-card__title"> 人脸 ${index + 1}</span>
            <span class="status-pill">${EXPRESSION_LABELS[topExpression.key]}</span>
          </div>
          <p class="result-card__highlight">
            主表情: <strong>${EXPRESSION_LABELS[topExpression.key]}</strong>
            (${formatPercent(topExpression.score)})
          </p>
          <ul class="result-list">${metrics}</ul>
        </article>
      `;
    })
    .join("");
}

function renderWebcamResults(detections, force = false) {
  const markup = buildResultsMarkup(detections);
  const now = Date.now();

  if (!force && markup === lastResultsMarkup) {
    return;
  }

  if (!force && now - lastResultsRenderAt < SIDEBAR_REFRESH_MS) {
    return;
```

```
  }

  webcamResults.innerHTML = markup;
  lastResultsMarkup = markup;
  lastResultsRenderAt = now;
}
```

摄像头启动函数兼容了现代 navigator.mediaDevices.getUserMedia 和旧版浏览器前缀接口。脚本会先显示等待授权状态，在用户允许访问摄像头后把视频流写入 video.srcObject；如果浏览器不支持或授权失败，则更新状态区和结果区提示。

```
function startVideo() {
  renderStatus(" 等待授权", " 未连接", " 允许浏览器访问摄像头后开始实时识别。");
  renderEmptyResults(
    " 等待识别",
    " 摄像头开始工作后，这里会同步显示表情识别结果副本。",
    true
  );

  const legacyGetUserMedia =
    navigator.getUserMedia ||
    navigator.webkitGetUserMedia ||
    navigator.mozGetUserMedia;

  const getUserMedia =
    navigator.mediaDevices && navigator.mediaDevices.getUserMedia
      ? (constraints) => navigator.mediaDevices.getUserMedia(constraints)
      : legacyGetUserMedia
      ? (constraints) =>
          new Promise((resolve, reject) => {
            legacyGetUserMedia.call(navigator, constraints, resolve, reject);
          })
      : null;

  if (!getUserMedia) {
    console.error("Camera API is not supported in this browser.");
    renderStatus(" 摄像头不可用", " 不支持", " 当前浏览器不支持摄像头访问接口。");
    renderEmptyResults(
      " 无法开始识别",
      " 请更换支持摄像头访问的浏览器后重试。",
      true
    );
    return;
  }

  renderStatus(" 正在连接", " 连接中", " 正在请求摄像头权限，请稍候。");

  getUserMedia({ video: {} })
    .then((stream) => {
      video.srcObject = stream;
      renderStatus(" 摄像头已连接", " 已连接", " 画面已连接，等待浏览器开始播放视频流。");
    })
    .catch((error) => {
      console.error(error);
      renderStatus(
        " 访问失败",
```

```
        " 不可用",
        " 未能获取摄像头权限，或者当前设备不可用。"
      );
      renderEmptyResults(
        " 无法开始识别",
        " 请检查浏览器授权和摄像头设备后重试。",
        true
      );
    });
}
```

当视频开始播放时，脚本创建覆盖画布并启动定时器。每轮定时任务都会同步画布尺寸，对当前视频帧执行人脸检测、关键点定位和表情识别，再清空上一帧标注并重绘最新检测框、关键点和表情标签。视频暂停或结束时，脚本及时清除定时器，避免后台继续执行无意义的推理。

```
video.addEventListener("play", () => {
  if (!canvas) {
    canvas = faceapi.createCanvasFromMedia(video);
    trackingStage.append(canvas);
  }

  syncCanvasSize();
  trackingStage.classList.add("stage-live");
  renderStatus(" 正在识别", " 识别中", " 已连接摄像头，正在同步更新实时识别结果。");

  window.clearInterval(detectionInterval);
  detectionInterval = window.setInterval(async () => {
    const displaySize = syncCanvasSize();
    const detections = await faceapi
      .detectAllFaces(video, new faceapi.TinyFaceDetectorOptions())
      .withFaceLandmarks()
      .withFaceExpressions();

    const resizedDetections = faceapi.resizeResults(detections, displaySize);
    canvas.getContext("2d").clearRect(0, 0, canvas.width, canvas.height);
    faceapi.draw.drawDetections(canvas, resizedDetections);
    faceapi.draw.drawFaceLandmarks(canvas, resizedDetections);
    faceapi.draw.drawFaceExpressions(canvas, resizedDetections);
    renderWebcamResults(resizedDetections);
  }, 100);
});

window.addEventListener("resize", () => {
  syncCanvasSize();
});

["pause", "ended"].forEach((eventName) => {
  video.addEventListener(eventName, () => {
    window.clearInterval(detectionInterval);
    renderStatus(" 识别已暂停", " 已暂停", " 摄像头画面已暂停，左侧结果已停止更新。");
    renderEmptyResults(
      " 识别已暂停",
      " 恢复摄像头画面后，这里会重新同步显示识别结果。",
      true
    );
  });
});
```

这段代码中的 newfaceapi.TinyFaceDetectorOptions() 表明实时检测采用轻量级检测器。每 100 ms 执行一次推理大致相当于每秒最多 10 次检测，实际帧率还会受到设备性能、浏览器和摄像头分辨率影响。定时器中先调用 clearRect 清除上一帧绘制内容，再绘制新一帧结果，否则旧检测框会残留在画布上。

人脸匹配与参考样本加载代码

除表情识别、年龄性别估计和实时追踪外，浏览器项目还保留了基于参考样本的人脸匹配流程。该流程先加载人脸识别、关键点和检测模型，再读取已按人物姓名分组的参考图片，生成带标签的人脸描述子集合，最后构造 FaceMatcher 用于新图片的身份匹配。

```
const imageUpload = document.getElementById("upload");
const analysisStage = document.getElementById("analysis-stage");
const MODEL_URL = "./models";
const LABELED_IMAGE_PATH = "./public/img/labeled_images";

Promise.all([
  faceapi.nets.faceRecognitionNet.loadFromUri(MODEL_URL),
  faceapi.nets.faceLandmark68Net.loadFromUri(MODEL_URL),
  faceapi.nets.ssdMobilenetv1.loadFromUri(MODEL_URL),
]).then(start);

async function start() {
  const container = document.createElement("div");
  container.className = "analysis-stage__media";
  analysisStage.append(container);

  const labeledFaceDescriptors = await loadLabeledImages();
  const faceMatcher = new faceapi.FaceMatcher(labeledFaceDescriptors, 0.6);
```

图片上传后，脚本会对新图片执行人脸检测、关键点定位和描述子提取，再把每个检测结果交给 faceMatcher.findBestMatch 进行最近邻匹配。匹配结果不是模型直接输出的人名，而是由新图片描述子与参考描述子之间的距离比较得到。

```
    const detections = await faceapi
      .detectAllFaces(image)
      .withFaceLandmarks()
      .withFaceDescriptors();

    const resizedDetections = faceapi.resizeResults(detections, displaySize);
    const results = resizedDetections.map((detection) =>
      faceMatcher.findBestMatch(detection.descriptor)
    );

    results.forEach((result, index) => {
      const box = resizedDetections[index].detection.box;
      const drawBox = new faceapi.draw.DrawBox(box, {
        label: result.toString(),
      });
      drawBox.draw(canvas);
    });
```

参考样本加载函数体现了注册图片与训练数据的区别。脚本从 public/img/labeled_images 下读取若干人物目录，每个人物目录提供 5 张参考图片；这些图片只用于提取描述子并构造 LabeledFaceDescriptors，不会更新预训练模型参数。

```
function loadLabeledImages() {
  const labels = ["Chandler", "Joey", "Monica", "Phoebe", "Rachel", "Ross"];

  return Promise.all(
    labels.map(async (label) => {
      const descriptions = [];

      for (let i = 1; i <= 5; i += 1) {
        const img = await faceapi.fetchImage(
          `${LABELED_IMAGE_PATH}/${label}/${i}.jpg`
        );
        const detections = await faceapi
          .detectSingleFace(img)
          .withFaceLandmarks()
          .withFaceDescriptor();

        descriptions.push(detections.descriptor);
      }

      return new faceapi.LabeledFaceDescriptors(label, descriptions);
    })
  );
}
```

开源人脸识别模型训练代码

本项目网页端直接使用 face-api.js 的预训练权重，并没有在浏览器项目中重新训练深度网络。为了说明人脸识别模型的训练思想，本节摘录 dlib 官方开源训练示例 dnn_metric_learning_on_images_ex.cpp 的关键片段。该示例展示了如何使用 loss_metric 训练人脸嵌入模型，使同一身份的人脸向量距离更近，不同身份的人脸向量距离更远。

示例开头说明了深度度量学习在人脸识别中的作用：模型不直接把图像分类为固定身份，而是学习一个向量空间，再用距离或最近邻方法完成识别。

```
// The contents of this file are in the public domain. See
↪  LICENSE_FOR_EXAMPLE_PROGRAMS.txt
/*
    This is an example illustrating the use of the deep learning tools from the
    dlib C++ Library.  In it, we will show how to use the loss_metric layer to do
    metric learning on images.

    The main reason you might want to use this kind of algorithm is because you
    would like to use a k-nearest neighbor classifier or similar algorithm, but
    you don't know a good way to calculate the distance between two things.  A
    popular example would be face recognition.  There are a whole lot of papers
    that train some kind of deep metric learning algorithm that embeds face
    images in some vector space where images of the same person are close to each
    other and images of different people are far apart.  Then in that vector
```

```
    space it's very easy to do face recognition with some kind of k-nearest
    neighbor classifier.

    In this example we will use a version of the ResNet network from the
    dnn_imagenet_ex.cpp example to learn to map images into some vector space where
    pictures of the same person are close and pictures of different people are far
    apart.

    You might want to read the simpler introduction to the deep metric learning
    API, dnn_metric_learning_ex.cpp, before reading this example.  You should
    also have read the examples that introduce the dlib DNN API before
    continuing.  These are dnn_introduction_ex.cpp and dnn_introduction2_ex.cpp.

*/
```

训练数据按身份目录组织。顶层目录下的每个子目录表示一个人物身份，子目录内保存该人物的多张图片。加载函数遍历这些子目录，形成按身份分组的图片列表，为后续 mini-batch 采样做准备。

```
// We will need to create some functions for loading data.  This program will
// expect to be given a directory structured as follows:
//    top_level_directory/
//        person1/
//            image1.jpg
//            image2.jpg
//            image3.jpg
//        person2/
//            image4.jpg
//            image5.jpg
//            image6.jpg
//        person3/
//            image7.jpg
//            image8.jpg
//            image9.jpg
//
// The specific folder and image names don't matter, nor does the number of folders or
// images.  What does matter is that there is a top level folder, which contains
// subfolders, and each subfolder contains images of a single person.

// This function spiders the top level directory and obtains a list of all the
// image files.
std::vector<std::vector<string>> load_objects_list (
    const string& dir
)
{
    std::vector<std::vector<string>> objects;
    for (auto subdir : directory(dir).get_dirs())
    {
        std::vector<string> imgs;
        for (auto img : subdir.get_files())
            imgs.push_back(img);

        if (imgs.size() != 0)
            objects.push_back(imgs);
    }
    return objects;
}
```

mini-batch 采样函数继续从每个身份下随机选择图片，并生成与图片对应的整数标签。这个标签不是人物姓名，而是身份在 `objs` 外层向量中的索引。函数还进行了颜色扰动和图像抖动，并检查同一 batch 中图片尺寸是否一致。

```
77  // This function takes the output of load_objects_list() as input and randomly
78  // selects images for training.  It should also be pointed out that it's really
79  // important that each mini-batch contain multiple images of each person.  This
80  // is because the metric learning algorithm needs to consider pairs of images
81  // that should be close (i.e. images of the same person) as well as pairs of
82  // images that should be far apart (i.e. images of different people) during each
83  // training step.
84  void load_mini_batch (
85      const size_t num_people,      // how many different people to include
86      const size_t samples_per_id, // how many images per person to select.
87      dlib::rand& rnd,
88      const std::vector<std::vector<string>>& objs,
89      std::vector<matrix<rgb_pixel>>& images,
90      std::vector<unsigned long>& labels
91  )
92  {
93      images.clear();
94      labels.clear();
95      DLIB_CASSERT(num_people <= objs.size(), "The dataset doesn't have that many people
        ↪  in it.");
96      std::vector<bool> already_selected(objs.size(), false);
97      matrix<rgb_pixel> image;
98      for (size_t i = 0; i < num_people; ++i)
99      {
100         size_t id = rnd.get_random_32bit_number()%objs.size();
101         // don't pick a person we already added to the mini-batch
102         while(already_selected[id])
103             id = rnd.get_random_32bit_number()%objs.size();
104         already_selected[id] = true;
105         for (size_t j = 0; j < samples_per_id; ++j)
106         {
107             const auto& obj = objs[id][rnd.get_random_32bit_number()%objs[id].size()];
108             load_image(image, obj);
109             images.push_back(std::move(image));
110             labels.push_back(id);
111         }
112     }
113     // You might want to do some data augmentation at this point.  Here we do some
        ↪  simple
114     // color augmentation.
115     for (auto&& crop : images)
116     {
117         disturb_colors(crop,rnd);
118         // Jitter most crops
119         if (rnd.get_random_double() > 0.1)
120             crop = jitter_image(crop,rnd);
121     }
122     // All the images going into a mini-batch have to be the same size.  And really,
        ↪  all
123     // the images in your entire training dataset should be the same size for what we
        ↪  are
```

```
    // doing to make the most sense.
    DLIB_CASSERT(images.size() > 0);
    for (auto&& img : images)
    {
        DLIB_CASSERT(img.nr() == images[0].nr() && img.nc() == images[0].nc(),
            "All the images in a single mini-batch must be the same size.");
    }
}
```

网络结构部分定义了训练网络和测试网络。训练网络使用带批归一化的 ResNet 主体，输出层为 `fc_no_bias<128>`，损失层为 `loss_metric`；测试网络则把批归一化替换为固定仿射变换，便于推理阶段稳定使用。

```
template <template <int,template<typename>class,int,typename> class block, int N,
↪  template<typename>class BN, typename SUBNET>
using residual = add_prev1<block<N,BN,1,tag1<SUBNET>>>;

template <template <int,template<typename>class,int,typename> class block, int N,
↪  template<typename>class BN, typename SUBNET>
using residual_down =
↪  add_prev2<avg_pool<2,2,2,2,skip1<tag2<block<N,BN,2,tag1<SUBNET>>>>>>;

template <int N, template <typename> class BN, int stride, typename SUBNET>
using block  = BN<con<N,3,3,1,1,relu<BN<con<N,3,3,stride,stride,SUBNET>>>>>;


template <int N, typename SUBNET> using res       =
↪  relu<residual<block,N,bn_con,SUBNET>>;
template <int N, typename SUBNET> using ares      =
↪  relu<residual<block,N,affine,SUBNET>>;
template <int N, typename SUBNET> using res_down  =
↪  relu<residual_down<block,N,bn_con,SUBNET>>;
template <int N, typename SUBNET> using ares_down =
↪  relu<residual_down<block,N,affine,SUBNET>>;

// ----------------------------------------------------------------------------
↪  -----

template <typename SUBNET> using level0 = res_down<256,SUBNET>;
template <typename SUBNET> using level1 = res<256,res<256,res_down<256,SUBNET>>>;
template <typename SUBNET> using level2 = res<128,res<128,res_down<128,SUBNET>>>;
template <typename SUBNET> using level3 = res<64,res<64,res<64,res_down<64,SUBNET>>>>;
template <typename SUBNET> using level4 = res<32,res<32,res<32,SUBNET>>>;

template <typename SUBNET> using alevel0 = ares_down<256,SUBNET>;
template <typename SUBNET> using alevel1 = ares<256,ares<256,ares_down<256,SUBNET>>>;
template <typename SUBNET> using alevel2 = ares<128,ares<128,ares_down<128,SUBNET>>>;
template <typename SUBNET> using alevel3 =
↪  ares<64,ares<64,ares<64,ares_down<64,SUBNET>>>>;
template <typename SUBNET> using alevel4 = ares<32,ares<32,ares<32,SUBNET>>>;


// training network type
using net_type = loss_metric<fc_no_bias<128,avg_pool_everything<
                            level0<
                            level1<
                            level2<
                            level3<
```

```
                            level4<
                            max_pool<3,3,2,2,relu<bn_con<con<32,7,7,2,2,
                            input_rgb_image
                            >>>>>>>>>>>>;

// testing network type (replaced batch normalization with fixed affine transforms)
using anet_type = loss_metric<fc_no_bias<128,avg_pool_everything<
                            alevel0<
                            alevel1<
                            alevel2<
                            alevel3<
                            alevel4<
                            max_pool<3,3,2,2,relu<affine<con<32,7,7,2,2,
                            input_rgb_image
                            >>>>>>>>>>>>;
```

训练阶段创建 dnn_trainer，设置学习率和同步文件，并通过多个加载线程持续产生 mini-batch。每个 mini-batch 包含多个身份以及每个身份的多张图片，使模型可以同时学习同一身份和不同身份之间的距离关系。训练结束后，示例把网络清理并序列化保存为 metric_network_renset.dat。

训练器初始化代码如下。这里的 sgd(0.0001,0.9) 设置了权重衰减和动量，set_learning_rate(0.1) 设置初始学习率，set_synchronization_file 则用于周期性同步训练状态。

```
    net_type net;

    dnn_trainer<net_type> trainer(net, sgd(0.0001, 0.9));
    trainer.set_learning_rate(0.1);
    trainer.be_verbose();
    trainer.set_synchronization_file("face_metric_sync", std::chrono::minutes(5));
    // Keep this small so the course experiment can finish in a reasonable time.
    // For a real model, use much more data and a much larger threshold.
    trainer.set_iterations_without_progress_threshold(80);
    const unsigned long max_training_steps = 500;
```

数据加载线程通过 dlib::pipe 把图片和标签送入训练主线程。主线程只需要不断取出队列中的 batch 并调用 train_one_step，从而把耗时的图片读取、解码和增强工作与训练计算并行起来。

```
    // If you have a lot of data then it might not be reasonable to load it all
    // into RAM.  So you will need to be sure you are decompressing your images
    // and loading them fast enough to keep the GPU occupied.  I like to do this
    // using the following coding pattern: create a bunch of threads that dump
    // mini-batches into dlib::pipes.
    dlib::pipe<std::vector<matrix<rgb_pixel>>> qimages(4);
    dlib::pipe<std::vector<unsigned long>> qlabels(4);
    auto data_loader = [&qimages, &qlabels, &objs](time_t seed)
    {
        dlib::rand rnd(time(0)+seed);
        std::vector<matrix<rgb_pixel>> images;
        std::vector<unsigned long> labels;
        while(qimages.is_enabled())
```

```
        {
            try
            {
                load_mini_batch(5, 5, rnd, objs, images, labels);
                qimages.enqueue(images);
                qlabels.enqueue(labels);
            }
            catch(std::exception& e)
            {
                cout << "EXCEPTION IN LOADING DATA" << endl;
                cout << e.what() << endl;
            }
        }
    };
    // Run the data_loader from 5 threads.  You should set the number of threads
    // relative to the number of CPU cores you have.
    std::thread data_loader1([data_loader](){ data_loader(1); });
    std::thread data_loader2([data_loader](){ data_loader(2); });
    std::thread data_loader3([data_loader](){ data_loader(3); });
    std::thread data_loader4([data_loader](){ data_loader(4); });
    std::thread data_loader5([data_loader](){ data_loader(5); });


    // Here we do the training.  The original example stops only after the learning
    // rate becomes very small.  For this course experiment, we also cap the number
    // of training steps so the program reaches the save-and-verify stage quickly.
    while(trainer.get_learning_rate() >= 1e-4 && trainer.get_train_one_step_calls() <
    ↪  max_training_steps)
    {
        qimages.dequeue(images);
        qlabels.dequeue(labels);
        trainer.train_one_step(images, labels);
```

学习率降到阈值以下后，示例保存网络并关闭加载线程。保存模型前调用 `net.clean()` 是为了去掉训练时的临时缓存，使序列化文件更适合后续推理使用。

```
    cout << "training steps: " << trainer.get_train_one_step_calls() << endl;

    // Wait for training threads to stop
    trainer.get_net();
    cout << "done training" << endl;

    // Save the network to disk
    net.clean();
    serialize("metric_network_renset.dat") << net;

    // stop all the data loading threads and wait for them to terminate.
    qimages.disable();
    qlabels.disable();
    data_loader1.join();
    data_loader2.join();
    data_loader3.join();
```

训练完成后，示例把图片送入测试网络得到嵌入向量，并根据向量距离判断两张图片是否属于同一身份。这个过程与浏览器项目中的 `FaceMatcher` 思路一致：先得到人脸描述子，再依据距离阈值完成匹配。

```
// Now, just to show an example of how you would use the network, let's check how well
// it performs on the training data.
dlib::rand rnd(time(0));
load_mini_batch(5, 5, rnd, objs, images, labels);

// Normally you would use the non-batch-normalized version of the network to do
// testing, which is what we do here.
anet_type testing_net = net;

// Run all the images through the network to get their vector embeddings.
std::vector<matrix<float,0,1>> embedded = testing_net(images);

// Now, check if the embedding puts images with the same labels near each other and
// images with different labels far apart.
int num_right = 0;
int num_wrong = 0;
for (size_t i = 0; i < embedded.size(); ++i)
{
    for (size_t j = i+1; j < embedded.size(); ++j)
    {
        if (labels[i] == labels[j])
        {
            // The loss_metric layer will cause images with the same label to be less
            // than net.loss_details().get_distance_threshold() distance from each
            // other.  So we can use that distance value as our testing threshold.
            if (length(embedded[i]-embedded[j]) < testing_net.loss_details().get_distance_threshold())
                ++num_right;
            else
                ++num_wrong;
        }
        else
        {
            if (length(embedded[i]-embedded[j]) >= testing_net.loss_details().get_distance_threshold())
                ++num_right;
            else
                ++num_wrong;
        }
    }
}

cout << "num_right: "<< num_right << endl;
cout << "num_wrong: "<< num_wrong << endl;
```

最后的两两比较代码尤其值得注意。它把同标签图片距离小于阈值视为正确，把不同标签图片距离大于等于阈值视为正确。这与浏览器端最近邻匹配的逻辑一致：模型负责输出描述子，识别逻辑负责比较描述子距离并决定是否接受某个身份标签。

Unity 森林场景 URP Shader 修复代码

第三章中森林资源包导入 Unity 6 + URP 项目后出现大面积粉色材质，核心原因是原资源包自带的 `StandardNoCulling.shader` 使用旧 Built-in Render Pipeline 的

Surface Shader 写法，无法在 URP 中正常渲染。本节收录修改后的 Shader 文件。该文件将原 Shader 改写为 URP 可用的 HLSL 实现，并保留双面渲染、透明裁剪、基础贴图、颜色、光滑度和阴影投射等植被材质所需功能。

```
Shader "Fantasy Forest/StandardNoCulling"
{
    Properties
    {
        _Cutoff("Alpha Clip Threshold", Range(0, 1)) = 0.5
        _MainTex("Base Map", 2D) = "white" {}
        _Color("Base Color", Color) = (1, 1, 1, 1)
        _Glossiness("Smoothness", Range(0, 1)) = 0
    }

    SubShader
    {
        Tags
        {
            "RenderPipeline" = "UniversalPipeline"
            "RenderType" = "TransparentCutout"
            "Queue" = "AlphaTest"
        }

        Cull Off
        ZWrite On
        Blend One Zero

        Pass
        {
            Name "ForwardLit"
            Tags { "LightMode" = "UniversalForward" }

            HLSLPROGRAM
            #pragma target 3.0
            #pragma vertex vert
            #pragma fragment frag
            #pragma multi_compile _ _MAIN_LIGHT_SHADOWS _MAIN_LIGHT_SHADOWS_CASCADE
            ↪  _MAIN_LIGHT_SHADOWS_SCREEN
            #pragma multi_compile_fragment _ _SHADOWS_SOFT
            #pragma multi_compile_fog

            #include
            ↪  "Packages/com.unity.render-pipelines.universal/ShaderLibrary/Core.hlsl"
            #include "Packages/com.unity.render-pipelines.universal/ShaderLibrary/Ligh」
            ↪  ting.hlsl"

            TEXTURE2D(_MainTex);
            SAMPLER(sampler_MainTex);

            CBUFFER_START(UnityPerMaterial)
                float4 _MainTex_ST;
                half4 _Color;
                half _Cutoff;
                half _Glossiness;
            CBUFFER_END

            struct Attributes
            {
                float4 positionOS : POSITION;
```

```
45              float3 normalOS : NORMAL;
46              float2 uv : TEXCOORD0;
47          };

48          struct Varyings
49          {
50              float4 positionCS : SV_POSITION;
51              float2 uv : TEXCOORD0;
52              float3 normalWS : TEXCOORD1;
53              float3 positionWS : TEXCOORD2;
54              half fogFactor : TEXCOORD3;
55          };

56          Varyings vert(Attributes input)
57          {
58              Varyings output;
59              VertexPositionInputs positions =
                ↪  GetVertexPositionInputs(input.positionOS.xyz);
60              VertexNormalInputs normals = GetVertexNormalInputs(input.normalOS);

61              output.positionCS = positions.positionCS;
62              output.positionWS = positions.positionWS;
63              output.normalWS = normals.normalWS;
64              output.uv = TRANSFORM_TEX(input.uv, _MainTex);
65              output.fogFactor = ComputeFogFactor(output.positionCS.z);
66              return output;
67          }

68          half4 frag(Varyings input) : SV_Target
69          {
70              half4 mainTex = SAMPLE_TEXTURE2D(_MainTex, sampler_MainTex, input.uv);
71              clip(mainTex.a - _Cutoff);

72              InputData lightingInput = (InputData)0;
73              lightingInput.positionWS = input.positionWS;
74              lightingInput.normalWS = normalize(input.normalWS);
75              lightingInput.viewDirectionWS =
                ↪  GetWorldSpaceNormalizeViewDir(input.positionWS);
76              lightingInput.shadowCoord =
                ↪  TransformWorldToShadowCoord(input.positionWS);
77              lightingInput.fogCoord = input.fogFactor;
78              lightingInput.bakedGI = SampleSH(lightingInput.normalWS);

79              SurfaceData surface = (SurfaceData)0;
80              surface.albedo = mainTex.rgb * _Color.rgb;
81              surface.alpha = 1;
82              surface.metallic = 0;
83              surface.specular = 0;
84              surface.smoothness = _Glossiness;
85              surface.occlusion = 1;

86              return UniversalFragmentPBR(lightingInput, surface);
87          }
88          ENDHLSL
89      }

90      Pass
91      {
92          Name "ShadowCaster"
93          Tags { "LightMode" = "ShadowCaster" }
```

```
94              ZWrite On
95              ZTest LEqual
96              ColorMask 0
97              Cull Off

98              HLSLPROGRAM
99              #pragma target 3.0
100             #pragma vertex vert
101             #pragma fragment frag

102             #include
                ↪  "Packages/com.unity.render-pipelines.universal/ShaderLibrary/Core.hlsl"

103             TEXTURE2D(_MainTex);
104             SAMPLER(sampler_MainTex);

105             CBUFFER_START(UnityPerMaterial)
106                 float4 _MainTex_ST;
107                 half4 _Color;
108                 half _Cutoff;
109                 half _Glossiness;
110             CBUFFER_END

111             struct Attributes
112             {
113                 float4 positionOS : POSITION;
114                 float2 uv : TEXCOORD0;
115             };

116             struct Varyings
117             {
118                 float4 positionCS : SV_POSITION;
119                 float2 uv : TEXCOORD0;
120             };

121             Varyings vert(Attributes input)
122             {
123                 Varyings output;
124                 output.positionCS = TransformObjectToHClip(input.positionOS.xyz);
125                 output.uv = TRANSFORM_TEX(input.uv, _MainTex);
126                 return output;
127             }

128             half4 frag(Varyings input) : SV_Target
129             {
130                 half alpha = SAMPLE_TEXTURE2D(_MainTex, sampler_MainTex, input.uv).a;
131                 clip(alpha - _Cutoff);
132                 return 0;
133             }
134             ENDHLSL
135         }
136     }

137     FallBack "Hidden/Universal Render Pipeline/FallbackError"
138 }
```

Unity VR 移动系统关键代码

第三章中的森林场景移动能力并非来自森林资源包本身，而是来自 Unity VR 模板和 XR Interaction Toolkit。移动输入由 `XRIDefaultInputActions.inputactions` 定义，再由 `ControllerInputActionManager.cs` 管理连续移动、传送移动、连续转向、瞬间转向和 UI 滚动等输入动作。下面代码片段展示了脚本中保存的输入动作引用和移动模式开关。

```
51          [FormerlySerializedAs("m_TeleportModeActivate")]
52          InputActionReference m_TeleportMode;
53          [SerializeField]
54          [Tooltip("The reference to the action to cancel the teleport aiming mode for
            ↪  this controller.")]
55          InputActionReference m_TeleportModeCancel;
56          [SerializeField]
57          [Tooltip("The reference to the action of continuous turning the XR Origin with
            ↪  this controller.")]
58          InputActionReference m_Turn;
59          [SerializeField]
60          [Tooltip("The reference to the action of snap turning the XR Origin with this
            ↪  controller.")]
61          InputActionReference m_SnapTurn;
62          [SerializeField]
63          [Tooltip("The reference to the action of moving the XR Origin with this
            ↪  controller.")]
64          InputActionReference m_Move;
65          [SerializeField]
66          [Tooltip("The reference to the action of scrolling UI with this controller.")]
67          InputActionReference m_UIScroll;
68          [Space]
69          [Header("Locomotion Settings")]
70          [SerializeField]
71          [Tooltip("If true, continuous movement will be enabled. If false, teleport will
            ↪  be enabled.")]
72          bool m_SmoothMotionEnabled;
73          [SerializeField]
74          [Tooltip("If true, continuous turn will be enabled. If false, snap turn will be
            ↪  enabled. Note: If smooth motion is enabled and enable strafe is enabled on
            ↪  the continuous move provider, turn will be overriden in favor of strafe.")]
75          bool m_SmoothTurnEnabled;
```

当移动模式发生变化时，脚本通过 `UpdateLocomotionActions` 启用或禁用对应输入动作。若启用连续移动，则使用 `Move`；若关闭连续移动，则启用 `TeleportMode` 和 `TeleportModeCancel`，从而进入传送移动方式。转向方式也在连续转向和瞬间转向之间切换。

```
458         void UpdateLocomotionActions()
459         {
460             // Disable/enable Teleport and Turn when Move is enabled/disabled.
461             SetEnabled(m_Move, m_SmoothMotionEnabled);
462             SetEnabled(m_TeleportMode, !m_SmoothMotionEnabled);
463             SetEnabled(m_TeleportModeCancel, !m_SmoothMotionEnabled);
464             // Disable ability to turn when using continuous movement
465             SetEnabled(m_Turn, !m_SmoothMotionEnabled && m_SmoothTurnEnabled);
466             SetEnabled(m_SnapTurn, !m_SmoothMotionEnabled && !m_SmoothTurnEnabled);
467         }
```

连续移动方向由 DynamicMoveProvider.cs 处理。该脚本继承自 XR Interaction Toolkit 的 ContinuousMoveProvider，并定义移动方向可以相对于头显或手柄计算。这样在 VR 场景中，用户既可以按照视线方向前进，也可以按照控制器指向进行移动。

```
12      public class DynamicMoveProvider : ContinuousMoveProvider
13      {
14          /// <summary>
15          /// Defines which transform the XR Origin's movement direction is relative to.
16          /// </summary>
17          /// <seealso cref="leftHandMovementDirection"/>
18          /// <seealso cref="rightHandMovementDirection"/>
19          public enum MovementDirection
20          {
21              /// <summary>
22              /// Use the forward direction of the head (camera) as the forward direction
                ↪  of the XR Origin's movement.
23              /// </summary>
24              HeadRelative,
25              /// <summary>
26              /// Use the forward direction of the hand (controller) as the forward
                ↪  direction of the XR Origin's movement.
27              /// </summary>
28              HandRelative,
29          }
```

脚本在运行时创建一个组合后的 forwardSource，再根据左右手输入强度混合左右参考姿态，最后调用父类的 ComputeDesiredMove 完成实际位移计算。这一逻辑使森林场景中的移动方向能够根据用户当前的 VR 输入状态动态变化，而不是固定在世界坐标系中。

```
102         protected override void Awake()
103         {
104             base.Awake();
105             m_CombinedTransform = new GameObject("[Dynamic Move Provider] Combined
                ↪  Forward Source").transform;
106             m_CombinedTransform.SetParent(transform, false);
107             m_CombinedTransform.localPosition = Vector3.zero;
108             m_CombinedTransform.localRotation = Quaternion.identity;
```

```
109             forwardSource = m_CombinedTransform;
110         }
111 
112         /// <inheritdoc />
113         protected override Vector3 ComputeDesiredMove(Vector2 input)
114         {
115             // Don't need to do anything if the total input is zero.
116             // This is the same check as the base method.
117             if (input == Vector2.zero)
118                 return base.ComputeDesiredMove(input);
119 
120             // Initialize the Head Transform if necessary, getting the Camera from XR
                ↪  Origin
121             if (m_HeadTransform == null)
122             {
123                 var xrOrigin = mediator.xrOrigin;
124                 if (xrOrigin != null)
125                 {
126                     var xrCamera = xrOrigin.Camera;
127                     if (xrCamera != null)
128                         m_HeadTransform = xrCamera.transform;
129                 }
130             }
131 
132             // Get the forward source for the left hand input
133             switch (m_LeftHandMovementDirection)
134             {
135                 case MovementDirection.HeadRelative:
136                     if (m_HeadTransform != null)
137                         m_LeftMovementPose = m_HeadTransform.GetWorldPose();
138 
139                     break;
140 
141                 case MovementDirection.HandRelative:
142                     if (m_LeftControllerTransform != null)
143                         m_LeftMovementPose = m_LeftControllerTransform.GetWorldPose();
144 
145                     break;
146 
147                 default:
148                     Assert.IsTrue(false, $"Unhandled
                        ↪  {nameof(MovementDirection)}={m_LeftHandMovementDirection}");
149                     break;
150             }
151 
152             // Get the forward source for the right hand input
153             switch (m_RightHandMovementDirection)
154             {
155                 case MovementDirection.HeadRelative:
156                     if (m_HeadTransform != null)
157                         m_RightMovementPose = m_HeadTransform.GetWorldPose();
158 
159                     break;
160 
161                 case MovementDirection.HandRelative:
162                     if (m_RightControllerTransform != null)
163                         m_RightMovementPose =
164                         ↪  m_RightControllerTransform.GetWorldPose();
165 
166                     break;
```

```
155                 default:
156                     Assert.IsTrue(false, $"Unhandled
                        ↪ {nameof(MovementDirection)}={m_RightHandMovementDirection}");
157                     break;
158             }

159             // Combine the two poses into the forward source based on the magnitude of
                ↪ input
160             var leftHandValue = leftHandMoveInput.ReadValue();
161             var rightHandValue = rightHandMoveInput.ReadValue();

162             var totalSqrMagnitude = leftHandValue.sqrMagnitude +
                ↪ rightHandValue.sqrMagnitude;
163             var leftHandBlend = 0.5f;
164             if (totalSqrMagnitude > Mathf.Epsilon)
165                 leftHandBlend = leftHandValue.sqrMagnitude / totalSqrMagnitude;

166             var combinedPosition = Vector3.Lerp(m_RightMovementPose.position,
                ↪ m_LeftMovementPose.position, leftHandBlend);
167             var combinedRotation = Quaternion.Slerp(m_RightMovementPose.rotation,
                ↪ m_LeftMovementPose.rotation, leftHandBlend);
168             m_CombinedTransform.SetPositionAndRotation(combinedPosition,
                ↪ combinedRotation);

169             return base.ComputeDesiredMove(input);
```